



Universidad Nacional de Quilmes
Departamento de Ciencia y Tecnología
Licenciatura en Informática

**Sistema de Seguimiento Académico para gestionar
decisiones en Direcciones de Carreras Universitarias**

Alumno: Juan Ignacio Yegro <21872>

Directora: Dra. Gabriela Arévalo

Bernal, 20 de Julio de 2021

Resumen

Dentro del contexto universitario, las universidades nacionales generan a lo largo de su historia diferentes repositorios de datos. Éstos están relacionados a las carreras (desde su creación) y sus respectivos planes de estudios, las diferentes materias y docentes que las dictan en esas carreras y como datos fundamentales, todos los datos de los estudiantes (desde su ingreso), de sus inscripciones y sus notas.

La Universidad Nacional de Quilmes (UNQ) está enmarcada dentro de este contexto. Particularmente desde la creación de la Tecnicatura en Programación Informática y la Licenciatura en Informática, las direcciones de ambas carreras han obtenido todos los datos relacionados a sus estudiantes y han generado diferentes bases de datos “*ad-hoc*” para realizar diferentes análisis iniciales de la población de ambas carreras.

Estos análisis se realizan tanto durante el proceso de inscripciones como durante el transcurso de cada cuatrimestre.

Debido a la falta de una sistematización del uso de datos, aún cuando toda la información está disponible y analizada, todo este proceso de análisis es ineficiente y demora decisiones relevantes a la carrera, en lo que respecta tanto a estudiantes como a docentes.

El objetivo de este trabajo es el desarrollo de una aplicación de seguimiento académico de los estudiantes de las carreras de informática de la UNQ, que resuelva diferentes necesidades de las direcciones de ambas carreras que les permitan consultar y analizar los datos para tomar decisiones de gestión en busca de las mejoras del desarrollo de las mismas.

Tabla de Contenidos

1. Introducción	1
1.1. Contexto	1
1.2. Objetivos: General y Específicos	4
1.3. Metodología de desarrollo	5
1.4. Estructura	5
2. Estado del Arte	6
2.1. Crecimiento exponencial de los datos	6
2.2. El rol de las universidades con respecto a los datos	6
2.2.1. Universidad de San Francisco	7
2.2.2. Universidad de Missouri	7
2.2.3. Universidad Estatal de Georgia	7
2.3. Trabajos relacionados	8
2.3.1. eCoach	8
2.3.2. Planteamiento del sistema de gestión académica orientado a serverless y microservicios	8
2.3.3. Sistema de Seguimiento Académico de Alumnos Universitarios - Universidad Nacional de Formosa	9
2.4. Contexto de las Universidades en Argentina	9
2.5. El rol de la Universidad Nacional de Quilmes	11
2.6. Planillas de datos	11
2.6.1. Planes de estudio	11
2.6.2. Datos personales	12
2.6.3. Materias cursadas	12
2.6.4. Inscripciones	14
2.7. Análisis de requerimientos	14
2.8. Conclusiones	16
3. Análisis de tecnologías	17
3.1. Python	17
3.1.1. Es interpretado	17
3.1.2. Es orientado a objetos	18

3.1.3. Es portable	18
3.1.4. Es extensible	18
3.1.5. Es de scripting	18
3.1.6. Es web	18
3.2. Django	19
3.2.1. ORM	20
3.3. Flask	21
3.4. Python en Data Science	21
3.4.1. Numpy	22
3.4.2. Pandas	22
3.5. Javascript	23
3.5.1. React	23
3.6. Microservicios	26
3.6.1. Heterogeneidad Tecnológica	26
3.6.2. Independencia	27
3.6.3. Escalamiento	27
3.6.4. Facilitar deploy	28
3.6.5. Reemplazabilidad	28
3.7. REST	29
3.7.1. Descripción de los recursos	29
3.7.2. Identificación de recursos	29
3.7.3. Manejo de errores	30
3.7.4. Uso de parámetros	30
3.7.5. Interacción con los recursos	31
3.7.6. REST y HTTP	32
3.8. JSON Web Tokens (JWT)	32
3.8.1. ¿Qué es?	32
3.8.2. ¿Cuándo usarlo?	32
3.8.3. Estructura	33
3.8.4. Header	33
3.8.5. Payload	33
3.8.6. Firma	34
3.8.7. Poniendo todo junto	34
3.8.8. ¿Cómo funciona?	35
3.8.9. Problemas que resuelve	35
3.8.10 Client-side/Stateless Sessions	35
3.8.11 ¿Es útil tener una sesión del lado del cliente?	36
3.8.12 Tokens de acceso y refresh	36
3.9. Docker	37
3.9.1. Introducción	37
3.9.2. Imágenes y contenedores	37

3.9.3. Contenedor	38
3.9.4. Docker Compose	40
3.9.5. Docker Swarm	40
3.10 Nginx	41
3.10.1 Forward proxy vs reverse proxy	41
3.10.2 Forward Proxy	41
3.10.3 Reverse Proxy	42
3.10.4 Balance de carga	42
3.11 Conclusiones	43
4. Desarrollo de la Solución	44
4.1. Introduccion	44
4.2. Núcleo	45
4.2.1. Tecnologías	45
4.2.2. Administración de los datos	46
4.2.3. Importadores	50
4.2.4. API	50
4.2.5. Tests	53
4.2.6. Integración Continua	54
4.2.7. Deploy	54
4.3. Análisis y Visualización de Datos	55
4.3.1. Análisis de datos	55
4.3.2. Visualización de los datos analizados	57
4.4. Licencia	74
4.5. Conclusiones	74
5. Pruebas realizadas	76
5.1. Pruebas y resultados	76
5.1.1. Resultados esperados	76
5.1.2. Pruebas con 100 usuarios simultáneos	77
5.1.3. Pruebas con 300 usuarios simultáneos	78
5.1.4. Pruebas con 300 usuarios simultáneos con dos instancias	79
5.2. Conclusiones sobre las pruebas	80
6. Conclusiones	82
6.1. Conclusiones	82
6.2. Lineas de Trabajos futuros	83
6.2.1. Extensión de análisis de datos	83
6.2.2. Aplicación para estudiantes	83
6.2.3. Integración con Guaraní	83
6.2.4. Integración con sistema de encuestas de inscripción	83
6.2.5. GraphQL en el núcleo	84

6.2.6. Proveedor de identidad (IDP) 84

6.2.7. Acceso Centralizado 84

Índice de figuras

3.1. Flujo de trabajo de Django	19
3.2. <i>ORM</i>	20
3.3. Flujo de datos unidireccional	25
3.4. Microservicios: Homogeneidad Tecnológica	27
3.5. Microservicios: escalamiento	28
3.6. Estructura de contenedores	39
3.7. Estructura de Máquinas virtuales	39
3.8. Funcionamiento de Forward Proxy	42
3.9. Funcionamiento de Reverse Proxy	42
4.1. Obtención de datos a través del núcleo	45
4.2. Logo del Núcleo	46
4.3. Pantalla de login	47
4.4. Pantalla principal del admin	47
4.5. Pantalla de importadores	48
4.6. Pantalla de listados	48
4.7. Pantalla de edición	49
4.8. Funcionamiento del pedido de token y posteriores requests	53
4.9. Test Coverage de 87,44 %	54
4.10Administración de contenedores desde Portainer	55
4.11Obtención de datos a través del núcleo	56
4.12Test Coverage de 90,1 %	57
4.13Logo de la aplicación de Seguimiento Académico	57
4.14Login de Seguimiento Académico	59
4.15Pantalla principal de Seguimiento Académico	60
4.16Pantalla Reporte de Carrera de Seguimiento Académico	60
4.17Pantalla Reporte de Materia de Seguimiento Académico	61
4.18Pantalla Reporte de Alumno de Seguimiento Académico	61
4.19Estadísticas básicas de la materia	62
4.20Detalle de aprobación de la materia	63
4.21Dispersión de notas con respecto al promedio	64

4.22	Recursantes	64
4.23	Porcentajes de aprobación por área	65
4.24	Porcentajes de aprobación por núcleo	66
4.25	Notas del estudiante	67
4.26	Porcentaje de avance de carrera	68
4.27	Detalle de aprobación	68
4.28	Ingresos por semestre	69
4.29	Tabla de estudiantes por cohorte	70
4.30	Tabla de ingresantes por cohorte	70
4.31	Cantidad de cursantes	71
4.32	Cantidad de ingresantes	71
4.33	Cantidad de graduados	72
4.34	Gráfico de dispersión	73
4.35	Tabla de ingresantes por cohorte	74

Índice de cuadros

1.1. Crecimiento de la Matrícula en la Licenciatura en Informática	2
1.2. Crecimiento de la Matrícula en la Tecnicatura en Programación In- formática	3
2.1. Ejemplo de una planilla de datos de un plan de estudios	11
2.2. Ejemplo de una planilla de datos personales de los estudiantes	12
2.3. Ejemplo de una planilla de inscripciones de estudiantes	14
3.1. Características de los métodos HTTP más comunes	32
4.1. Método para pedir un token al núcleo	51
4.2. Métodos para pedir datos. (cc: código de carrera, pa: plan año)	51
4.3. Áreas de la Licenciatura en Informática	66
4.4. Núcleos de la Licenciatura en Informática	67
5.1. Pruebas realizadas con 100 usuarios simultáneos durante 5 minutos	77
5.2. Recursos en promedio consumidos por el contenedor durante la prue- ba	77
5.3. APDEX de la prueba con 100 usuarios durante 5 minutos	78
5.4. Pruebas realizadas con 300 usuarios simultáneos durante 10 minutos	78
5.5. Recursos en promedio consumidos por el contenedor durante la prue- ba	79
5.6. APDEX de la prueba con 300 usuarios durante 10 minutos y una instancia de la aplicación	79
5.7. Pruebas realizadas con 300 usuarios simultáneos durante 10 minutos	80
5.8. Recursos en promedio consumidos por los contenedores durante la prueba	80
5.9. APDEX de la prueba con 300 usuarios durante 10 minutos y dos ins- tancias de la aplicación	80

Capítulo 1

Introducción

1.1. Contexto

El déficit en ingenieros y graduados en las llamadas disciplinas STEM (Ciencia, Tecnología, Ingeniería y Matemática) es un problema que no sólo afecta a nuestro país, sino que se manifiesta como una problemática global. En el 2016¹, particularmente en Argentina, se reportaba que cada año quedaban sin cubrir 5.000 puestos en la industria del software por falta de profesionales (según la Cámara de la Industria Argentina del Software²). El sector empleaba a 90.000 personas y representaba una de las principales exportaciones de valor agregado con un crecimiento del 10 % anual, pero “la matrícula en carreras de sistemas quedó estancada en 20.000 y se recibían 4.000 por año, cuando la industria requiere el doble”(según Fundación Sadosky³). Aún en contextos actuales (afectados fuertemente por la pandemia), se mantiene un de escasez de talento en puestos de empresas tecnológicas⁴. Bajo este contexto, se implementaron la Tecnicatura en Programación Informática (TPI) en 2007 (por la Resolución Consejo Superior 091-07⁵ de la UNQ) y la Licenciatura en Informática (LI) en 2012 (por la Resolución Consejo Superior 190-12⁶) en la Universidad Nacional de Quilmes (UNQ) para desarrollar profesionales altamente calificados y ofrecer carreras distintivas de aquellas ofertadas por los centros universitarios próximos (UBA y UNLP, entre otras) y que apunten a cubrir necesidades sociales e industriales

¹<https://www.uba.ar/noticiasuba/nota.php?id=15732>

²<http://www.cessi.org.ar>

³<http://www.fundacionsadosky.org.ar/>

⁴https://www.clarin.com/economia/record-escasez-talento-perfiles-dificiles-encontrar_0_350M4Uyzd.html

⁵http://cpi.blog.unq.edu.ar/wp-content/uploads/sites/14/2021/08/2007.RCS_091_07_Plan_TPI_2007.pdf

⁶http://cpi.blog.unq.edu.ar/wp-content/uploads/sites/14/2021/08/2012.RCS_190_12_Creacion_Plan_Estudios_2012_LI.pdf

concretas descriptas previamente.

Desde la creación de ambas carreras, se ha dado un crecimiento constante en sus respectivas matrículas calculadas a partir de dos medidas relevantes: el número de estudiantes que ingresaban al Curso de Ingreso (que no eran considerados como estudiantes regulares de las carreras) y los estudiantes que estaban activos en las carreras. Esta situación fue dada hasta el 2015.

A partir del 2015, el crecimiento es más notable en la UNQ pues se incorporaron a los planes de estudio las materias del *Curso de Ingreso* como parte de las materias de la carrera, denominándolo como *Ciclo Introductorio* (CI). A partir de este cambio, los estudiantes ya eran parte de la carrera cuando se incorporaban a la UNQ y los números de ambos grupos se unificaron en una sola medida. Las Tablas 1.1 y 1.2 evidencian ese crecimiento mencionado en números. Los números de las tablas se obtuvieron a partir de informes internos que las Direcciones de Carrera (TPI y LI) recibe de parte de la Secretaría de Gestión Académica.

Año	Estudiantes	Total Matrícula	Tasa Crecimiento
2013	23	23	
2014	94	117	5,09
2015	97	214	1,83
2016	333	547	2,56
2017	323	870	1,59
2018	297	1167	1,34
2019	384	1551	1,33
2020	349	1900	1,23
2021	306	2206	1,16

Tabla 1.1: Crecimiento de la Matrícula en la Licenciatura en Informática

Desde sus inicios, la dirección de ambas carreras ha creado y mantenido un registro de datos personales de los estudiantes, las materias cursadas y las materias que tienen intenciones de cursar cuatrimestre a cuatrimestre de los estudiantes que hayan aprobado al menos 2 de las 3 materias del CI. Gracias a este registro se puede realizar determinadas tareas:

- Se organizan las intenciones de inscripción de cada estudiante.
- Se generan las listas de emails con las cuales trabaja cada materia.
- Se maneja el cupo de las comisiones de las materias para evitar superpoblación en las mismas.
- Se detectan posibles candidatos a presentarse como auxiliares académicos.

Año	Estudiantes	Total Matrícula	Tasa Crecimiento
2007	49	49	
2008	106	155	3,16
2009	99	254	1,64
2010	108	362	1,43
2011	142	504	1,39
2012	112	616	1,22
2013	166	782	1,27
2014	103	885	1,13
2015	88	973	1,10
2016	287	1260	1,29
2017	295	1555	1,23
2018	271	1826	1,17
2019	383	2209	1,21
2020	376	2585	1,17
2021	391	2976	1,15

Tabla 1.2: Crecimiento de la Matrícula en la Tecnicatura en Programación Informática

- Se detectan diferentes problemáticas en la cursada de los estudiantes (sobretudo en las instancias finales de la carrera).
- Se realizan diferentes análisis de la evolución de la carrera de los estudiantes durante cada cuatrimestre

En la actualidad, todos los análisis descritos previamente se realizan en base a repositorios “ad-hoc” (formados por planillas de cálculo en formato .ods) que mantienen toda la información de los estudiantes y materias. Esta información se obtuvo ya sea a través de los estudiantes mismos o bien, usando el sistema SIU-Guaraní de la UNQ, a través de informes pedidos explícitamente por la Dirección de Carreras. Con el incremento de la matrícula año a año, este proceso de análisis manual se hace más complejo y lleva demasiado tiempo. Las direcciones de las carreras de informática trabajan alrededor de una semana a tiempo completo pasado el período de inscripciones a materias y terminadas las cursadas para hacer un procesamiento adecuado con la información de los estudiantes (sobretudo quienes comenzaron el Ciclo Básico de cualquiera de las carreras). Este último grupo de estudiantes es relevante porque hasta tanto el estudiante no haya completado parcialmente el CI, sus datos no los registra la Dirección de Carreras.

Durante el cuatrimestre, el sistema de registros se consulta constantemente para generar diferentes estadísticas para planificar la evolución de la carrera. Aún cuando toda la información está disponible y analizada, todo este proceso

es manual, es ineficiente y demora decisiones relevantes a la carrera, en lo que respecta tanto a estudiantes como a docentes. Los problemas actuales (entre otros) son:

- Como la información se guarda en un repositorio “*ad-hoc*”, no existe la posibilidad de tener varios usuarios (directores, asistentes de carrera, docentes) colaborando en forma cooperativa (distribuida).
- Los datos provistos por SIU-Guaraní y los resultados de cada materia al finalizar cada cuatrimestre se incorporan de forma manual en los registros.
- Los análisis se realizan con fórmulas analizando las planillas *.ods* y su información (ejemplo, generación de estadísticas).
- No existe una forma automática de controlar la información que tiene SIU-Guaraní acerca de los estudiantes y su situación de sus cursadas en las carreras. En la actualidad, la única forma de acceder a los datos de Guaraní es a través del pedido explícito de informes de la Dirección de Carreras a la Secretaría de Gestión Académica, que es la oficina responsable de la administración de SIU-Guaraní.
- No se pueden generar estadísticas relevantes debido al análisis manual de los datos.

1.2. Objetivo general

En base a esta lista de problemas que se enumeraron en la Sección 1.1, este trabajo plantea como objetivo general el desarrollo de una aplicación de seguimiento académico que contribuya en el proceso de gestión y decisión de la Dirección de Carreras de TPI/LI, en las siguientes acciones:

- Analizar información de los estudiantes y de las materias que cursa o debe cursar de la TPI y de la LI, desde su incorporación a la carrera hasta el final de la carrera
- Generar recomendaciones para los estudiantes acerca de sus elecciones de materia a principios de cada cuatrimestre
- Generar estadísticas y visualizaciones relevantes que ayuden a la Dirección de Carreras a tomar decisiones durante el año académico.

1.3. Metodología de desarrollo

Para el desarrollo de la aplicación de seguimiento académico, realizaremos en forma iterativa las siguientes etapas:

- Relevamiento de los requerimientos y casos de uso.
- Relevamiento del hardware disponible en la Universidad.
- Diseño del software y la arquitectura del mismo.
- Desarrollo de software.
- Tests de unidad e integración.
- Pruebas de stress.
- Documentación.
- Puesta en producción.

1.4. Estructura de la Tesis

La estructura de la presente tesis es la siguiente. El presente Capítulo (1) introduce brevemente el contexto general del trabajo, y se presentan los objetivos y la metodología de desarrollo de la aplicación de Seguimiento Académico. El Capítulo 2 muestra el estado del arte con diferentes análisis de otras universidades, menciona los trabajos relacionados, cuáles son las fuentes de datos que se usan en la aplicación desarrollada y cuáles son los requerimientos de la aplicación desarrollada. El Capítulo 3 analiza detalladamente las diferentes tecnologías usadas en el desarrollo. El Capítulo 4 muestra en detalle el desarrollo del núcleo, el análisis y visualización que provee la aplicación. El Capítulo 5 muestra cuáles son las diferentes pruebas realizadas a la aplicación para testear su funcionamiento. Finalmente, el Capítulo 6 concluye este trabajo y detalla líneas de trabajos futuros para agregar funcionamiento a la aplicación desarrollada.

Capítulo 2

Estado del Arte

2.1. Crecimiento exponencial de los datos

La humanidad, hasta el año 2005, creó 130 ExaBytes de datos (un exabyte son 1.073.741.824 GB). Para el año 2010, la cantidad de datos subió a 1200 exabytes. Para el año 2015, la cantidad de datos subió a 7900 exabytes. Y para el año 2020, se estima que esa cantidad llegó a los 40900 exabytes [IDC]. No existen estimaciones documentadas de la cantidad de datos generados entre el 2020 y 2021 en este nuevo contexto de pandemia.

En resumen, la cantidad de datos que producimos los humanos está creciendo exponencialmente, y las universidades no son la excepción.

2.2. El rol de las universidades con respecto a los datos

Hoy en día, el uso de los datos de los estudiantes para propósitos académicos se está transformando más en una norma que en una excepción en las universidades del mundo. Estos datos se usan para ayudar a los estudiantes a tener éxito, o para ofrecerles ayuda cuando se considera que no están rindiendo de la forma esperada. El análisis de los datos se hace cada vez más frecuente y necesario, y puede ser más eficiente ya que las herramientas de análisis fueron mejorando durante la última década.

En las secciones siguientes se describen brevemente el caso de 3 universidades que publicaron sus estrategias de análisis con respecto a sus estudiantes.

2.2.1. Universidad de San Francisco

En la Universidad de San Francisco (USF) [She20a], los datos de los estudiantes tales como asistencia, notas, y materias son usados para determinar su progreso. Cuando observan que un estudiante está teniendo dificultades, usando la tecnología desarrollada por el departamento de sistemas de USF, alertan a los tutores para que hablen con el estudiante. Además, usan una aplicación móvil llamada *USFMobile*¹, que consulta datos de sus sistemas y generan alertas.

2.2.2. Universidad de Missouri

La Universidad de Missouri [She20a] arma cuestionarios de 12 preguntas para los estudiantes. El propósito es conocer cómo los estudiantes se desenvuelven en términos académicos, sociales y financieros. Los datos se almacenan en los sistemas de la universidad. Los tres temas principales que mostró la encuesta, se relacionaron con las dificultades de algunos cursos, la asistencia y las dificultades financieras de los estudiantes. Estos datos le dieron a la Universidad un panorama de la situación de los estudiantes y puede responder rápidamente en consecuencia.

2.2.3. Universidad Estatal de Georgia

La Universidad Estatal de Georgia ² se convirtió en un ejemplo a nivel nacional en Estados Unidos, por el papel que adaptó para ayudar a los estudiantes de bajos recursos y los de primera generación. Esta universidad usa análisis predictivos para identificar a los estudiantes que podrían abandonar o no llegar a recibirse. Esto lo logran usando información pasada, la cual es utilizada para predecir futuros eventos. La Universidad de Georgia analizó 2 millones y medio de calificaciones obtenidas por los estudiantes a lo largo de 10 años para crear una lista de factores que predecían qué estudiantes son menos probables a graduarse. Este sistema tiene más de 800 alertas, apuntadas a indicarle a los tutores acerca de sus estudiantes y así poder ayudarlos. Por ejemplo, un tutor recibe un alerta cuando un estudiante no recibe una nota satisfactoria en una materia que es fundamental para su formación.

¹<https://www.usfca.edu/mobile>

²<https://www.lonestar.edu/multimedia/ThePromiseandPerilofPredictiveAnalyticsinHigherEducation.pdf>

2.3. Trabajos relacionados

2.3.1. eCoach

*ECoach*³ es una aplicación diseñada en la Universidad de Michigan para estudiantes de primer año que toman clases de ciencia, tecnología, ingeniería o matemática. Esta aplicación toma datos que las universidades ya tienen, como las materias que cursó un estudiante, las notas, etc. Otro aspecto importante de esta aplicación, es que se recibe *feedback* de parte de los estudiantes con respecto a materias, profesores, horarios, etc.

2.3.2. Planteamiento del sistema de gestión académica orientado a *serverless* y microservicios

El trabajo “Planteamiento del sistema de gestión académica orientado a *serverless* y microservicios de la Universidad Distrital F.J.D.C orientado a *serverless* y micro servicios”⁴ se desarrolló en la Universidad Distrital Francisco José de Caldas de Bogotá (Colombia). Este trabajo contempla el desarrollo de una aplicación que implica diferentes aspectos:

- La asignación y evaluación de docentes en las materias.
- La generación de reportes de evolución.
- La gestión de movilidad de estudiantes de intercambio.
- La gestión de manejo de notas.
- La gestión de inscripciones comunes y basadas en tutorías.
- La gestión de tutorías.
- La administración de planes de estudios, de exámenes y de trabajos finales.

El alcance del trabajo es muy amplio si se compara con este trabajo. La diferencia es que solamente se presentan los modelos de las arquitecturas, sin una implementación concreta del mismo.

³<https://ecoach.ai.umich.edu/Welcome/>

⁴<https://github.com/gerardoRolong/ModelamientoArquitecturalAcademica>

2.3.3. Sistema de Seguimiento Académico de Alumnos Universitarios, Universidad Nacional de Formosa

El trabajo “Sistema de Seguimiento Académico de Alumnos Universitarios” de la materia Programación III de la Universidad Nacional de Formosa ⁵ es un proyecto que tiene como objetivo capturar el rendimiento académico y la situación en la que se encuentra cada estudiante, a partir de un puntaje calculado por un sistema mediante los datos almacenados en una base de datos diseñada especialmente para el sistema en sí. El objetivo final es detectar a aquellos estudiantes con mayor probabilidad de dejar la carrera, de acuerdo al puntaje obtenido, o con mayor probabilidad de que se “estancue”; debido a fallos en el transcurso de la misma. El cálculo se realiza en base a las siguientes variables:

- Antecedentes académicos.
- Información del desempeño actual de los estudiantes.
- Antecedentes socio-económicos y personales.
- Razones de dispersión.

En el caso particular de este trabajo, es similar a lo planteado, pero solo busca un índice para la evaluación de los estudiantes. En este trabajo, buscamos que la aplicación genere diferentes índices y resultados que ayuden a diversas decisiones que puede tomar la Dirección de las carreras de la UNQ. Aún cuando es un trabajo realizado por estudiantes, se consideró relevante como parte del estado del arte por la descripción similar al trabajo que se presenta en este trabajo.

2.4. Contexto de las Universidades en Argentina

Las Universidades Nacionales (públicas o privadas) guardan sus datos a través del Sistema de Información Universitaria (<http://www.siu.edu.ar>).

Este sistema se compone de un grupo de aplicaciones informáticas que colaboran en la gestión y la calidad de los datos que se producen en el ámbito de cada universidad.

El SIU tiene en la actualidad las siguientes aplicaciones:

- SIU-Guaraní (utilizado por UNQ)
- SIU-Pilagá (utilizado por UNQ)
- SIU-Mapuche (utilizado por UNQ)

⁵<https://freesoft.dev/program/73303194>

- SIU-Diaguita (utilizado por UNQ)
- SIU-Kolla
- SIU-Sanavirón/Quilmes (utilizado por UNQ)
- SIU-Tehuelche (utilizado por UNQ)
- SIU-Wichi (utilizado por UNQ)
- SIU-Araucano
- SIU-Bibliotecas
- SIU-Toba (utilizado por UNQ)
- SIU-Arai / SIU-Huarpe (utilizado por UNQ)

De las aplicaciones enumeradas, se detallan a continuación aquellas que podrían contribuir con información adicional a nuestra aplicación en trabajos futuros:

- SIU Guaraní que se utiliza para la administración de las tareas académicas para obtener información consistente para niveles operativos y directivos. Las funcionalidades actuales se detallan en <http://www.siu.edu.ar/siu-guarani>
- SIU Kolla permite la generación de encuestas para realizar relevamientos sobre distintas problemáticas del ámbito universitario. Las funcionalidades actuales se detallan en <http://www.siu.edu.ar/siu-kolla>. En la actualidad, este módulo no está instalado en los servidores de la UNQ.
- SIU Araucano permite informar estadísticas de ingreso, regularidad y egreso de los estudiantes. Además, procesa las cifras de la oferta educativa, como las cantidades de alumnos por materia, materias aprobadas por alumno, materias ofertadas o la antigüedad de los alumnos. Las funcionalidades actuales se detallan en <http://www.siu.edu.ar/siu-araucano>. En la actualidad, este módulo no está instalado en los servidores de la UNQ.
- SIU Wichi permite visualizar y analizar de manera integrada los datos históricos de ejecución presupuestaria, académicos de personal y patrimonio, buscando como objetivo colaborar con las decisiones que tomen los distintos actores de la organización, sustentadas sobre una base de conocimiento. Las funcionalidades actuales se detallan en <http://www.siu.edu.ar/siu-wichi>

2.5. El rol de la Universidad Nacional de Quilmes

Como se mencionó previamente, la UNQ (como otras universidades) tiene en la actualidad todo el historial de las carreras, sus estudiantes, sus materias cursadas y las notas obtenidas. La Universidad también cuenta con otros datos referentes a su funcionamiento como institución educativa, pero sólo se mencionaron aquellos que son relevantes para el desarrollo de este informe y la aplicación correspondiente.

Todos los datos mencionados previamente que pertenecen al ámbito de la UNQ, están guardados en diferentes módulos del SIU (descriptos en la sección previa), los cuales están alojados en los servidores de la UNQ.

Se debe remarcar que los módulos de SIU generan datos o estadísticas generales de las carreras, y que nuestro trabajo tiene como propósito usar estos datos para tomar decisiones de las problemáticas propias de las carreras de informática de la UNQ.

2.6. Planillas de datos

La información solicitada a la Secretaría de Gestión Académica se obtiene a través de planillas *.ods*. A continuación se listan las diferentes planillas y sus formatos.

2.6.1. Planes de estudio

Los datos de planes de estudio contienen información de las materias detallando la carrera, el plan de la carrera a la cual pertenece la materia, el cuatrimestre en el cual está recomendada en el plan de estudios, el núcleo (o ciclo) de la carrera donde está agrupada la materia, el área donde está agrupada la materia, el código, los créditos y nombre de la materia.

Las columnas de la planilla presentan el formato detallado en la Tabla 2.1.

Carrera	Plan	Cuat.	Núcleo	Área	Materia	Créd.	Nombre
W	2019	3	B	Programación	1035	12	Base de Datos

Tabla 2.1: Ejemplo de una planilla de datos de un plan de estudios

Los datos de esta tabla reflejan los mismos datos que se especificaron en el plan de estudios de la carrera reportada.

2.6.2. Datos personales

La planilla de datos personales contiene la información con el formato detallado en la Tabla 2.3.

Legajo	DNI	Apellido	Nombre	Email	Fecha	Carrera	Plan
21872	35905769	Yegro	Juan	jy@unq.edu.ar	01-02-2009	W	2019

Tabla 2.2: Ejemplo de una planilla de datos personales de los estudiantes

Se debe remarcar que la fecha es el día en que el estudiante se inscribió en la carrera, y el plan en el cual se haya inscripto en el momento que se genera el acta. Es decir, si un estudiante cambió de plan desde su inscripción a los planes actuales (2019), se muestra este último plan.

Estos datos son útiles para que las direcciones de carrera puedan identificar a los estudiantes.

2.6.3. Materias cursadas

La planilla del historial de cursadas de una carrera determinada tiene el siguiente formato, donde cada fila representa la cursada de un estudiante en una materia determinada.

- **Legajo**
- **DNI**
- **Carrera:** Muestra el código de la Carrera
 - **P:** Tecnicatura en Programación Informática.
 - **W:** Licenciatura en Informática.
- **Regular:** Muestra la condición de regularidad del estudiante al momento de generar la planilla.
- **Calidad:** Muestra si el estudiante está:
 - **Activo:** Si el proceso de inscripción a la carrera se completó exitosamente.
 - **Pasivo:** Si el proceso de inscripción a la carrera no se completó exitosamente.
 - **Egresado:** Si el estudiante ha egresado de la carrera.
- **Materia:** Muestra el código de la Materia según el plan de estudios

- **Nombre:** Muestra el nombre de la Materia según el plan de estudios.
- **Fecha:** Muestra la fecha del Acta donde se registró la nota.
- **Resultado:** Muestra el resultado de la evaluación con los siguientes códigos:
 - **A:** Aprobado (por modalidad de mesa libre o equivalencia)
 - **E:** Pendiente Aprobación
 - **N:** Reprobado (por modalidad de promoción)
 - **P:** Aprobado (por modalidad de promoción)
 - **U o Vacío:** Ausente
 - **R:** Reprobado (por modalidad de mesa libre)
- **Nota:** Muestra la nota con:
 - Un valor numérico (de **1 a 10**).
 - **A** por Aprobado, en casos de materias particulares.
 - **PA** para Pendientes de Aprobación.
 - **Vacío** para Ausente.
- **Forma:** Muestra la forma de evaluación de la materia:
 - **Promoción:** para materias aprobadas al final de los cuatrimestres o bien en sus instancias de integradores válidos.
 - **Promoción en otra carrera:** para materias aprobadas (con el mismo código) en otras carreras distintas a la cual está inscripta el estudiante.
 - **Examen:** para materias aprobadas en fechas de mesas libres.
 - **Equivalencia:** para materias reconocidas por trámite de equivalencia.
 - **Examen Equivalente:** para materias rendidas en mesa libre en otra carrera.
 - **Equivalencia equivalente:** para materias reconocidas como Aprobada por equivalencia en otras carreras, o por cambios de plan de estudios o por cambios de carrera para materias.
- **Créditos:** Muestra los créditos de la materia.
- **Acta:** Muestra el número de acta de promoción (materias que se dictan durante cada cuatrimestre o instancia de integración). En caso contrario, muestra un 0.

- **Acta E:** Muestra el número de acta de mesa libre de materias. En caso contrario, es un campo vacío.
- **Plan:** Muestra el plan de la carrera donde está actualmente inscripto el estudiante.

2.6.4. Inscripciones

Esta planilla indica las inscripciones a una materia por parte de un estudiante.

Carrera	DNI	Legajo	Código de materia	Comisión	Fecha
W	35905769	21872	01036	C2	01/03/2016

Tabla 2.3: Ejemplo de una planilla de inscripciones de estudiantes

Se debe remarcar que la fecha es el día en que el estudiante se inscribió en la materia, y el plan en el cual se haya inscripto en el momento que se genera el acta. Es decir, si un estudiante cambió de plan desde su inscripción a los planes actuales (2019), se muestra este último plan.

2.7. Análisis de requerimientos

Luego de hacer un análisis de requerimientos con el equipo de Dirección de las distintas carreras de Ciencia y Tecnología (CyT) de la UNQ, se identificaron las siguientes necesidades:

- Calcular el avance de carrera, avance de diplomatura y avance de ciclo introductorio en base a los créditos.
- Calcular el porcentaje de materias aprobadas del total de la Carrera, el porcentaje de materias del Ciclo Básico, el porcentaje de materias del Ciclo Avanzado, el porcentaje de las Complementarias y el porcentaje del Ciclo Introductorio.
- Calcular el porcentaje de aprobación por área, que en el caso de la Licenciatura en Informática estas áreas son:
 - Programación.
 - Sistemas Informáticos.
 - Procesos Informáticos.
 - Desarrollo de Software.

- Teoría de la Computación.
- Calcular los pedidos específicos de CONEAU para acreditación de carreras:
 - Postulantes: cantidad de estudiantes que se anotaron.
 - Ingresantes: cantidad de estudiantes que cursaron al menos una materia del Ciclo Introductorio.
 - Estudiantes: cantidad de estudiantes activos.
 - Tabla de ingresantes por cohorte.
 - Tabla de cursantes por cohorte.
 - Tabla de graduados por cohorte.
 - Tasas de crecimiento anual y por cuatrimestre de las carreras, cursantes y solo ingresantes.
- Para una materia determinada conocer:
 - Cantidad de aprobados.
 - Cantidad de ausentes.
 - Cantidad de desaprobados.
- Calcular listados de estudiantes con una determinada materia aprobada, que distinga en: materia aprobada por equivalencia, examen libre, pendiente, acta de cursada común.
- Para los inscriptos a una materia generar listados de cuáles son recursantes, y cuántas veces.
- Realizar análisis retrospectivos y riesgos de deserción en base al historial general de la carrera.
- Identificar cuáles son las materias que demoran en la formación a los estudiantes en las carreras.
- Predecir inscripciones a carreras y materias.
- Analizar cumplimientos de prerrequisitos según plan de estudios.

2.8. Conclusiones

Al implementar una aplicación de **Seguimiento Académico** que se presenta en este trabajo, los datos estarán disponibles para ser consultados por las carreras. Además, existe un potencial para que esta solución pueda extenderse en el futuro y que esto implique una mejor experiencia también para los estudiantes. Llevar a cabo este proyecto implicará una mejora en cuanto a decisiones que se pueden tomar con respecto a los estudiantes en pos de una mejor calidad educativa. Esa calidad educativa puede darse con acciones como ayudar a determinar qué estudiantes están teniendo problemas, qué materias están trabando a los estudiantes y cuáles están cerca de recibirse.

Capítulo 3

Análisis de tecnologías

En base a los requerimientos especificados en el Capítulo 2, se decidió que la aplicación de Seguimiento Académico se desarrollaría en 3 partes. Por un lado, un núcleo que se encargase de manejar los datos y servirlos mediante una *API REST*. Luego, para el análisis y visualización de los datos se decidió separarlo en dos aplicaciones diferentes. La primera, desarrollada con *Flask*, que conjuntamente con *Pandas* y *NumPy* se encargará de analizar y servir los datos analizados. La segunda, desarrollada con *React*, tendrá la tarea de mostrar los datos analizados. Para la elección de este contexto tecnológico, en el presente Capítulo se explicará en detalle la descripción de todas las tecnologías involucradas que permitieron la implementación de la aplicación mencionada.

3.1. *Python*

Python es un lenguaje de programación de alto nivel, de propósito general e interpretado. Es un lenguaje *open-source* y por ende, el código fuente se puede obtener bajo la licencia *GNU General Public License (GPL)*. Python permite programar sistemas de diferentes envergaduras de forma clara y fácil. Su filosofía de diseño se concentra en la legibilidad, usando espacios en blanco. Más importante, *Python* incorpora los paradigmas de programación orientada a objetos, funcional y procedural con una biblioteca estándar muy enriquecida.

3.1.1. Es interpretado

Python es procesado en tiempo de ejecución por el intérprete. El programa entero no requiere compilación antes de ser ejecutado. El intérprete directamente ejecuta el programa, línea por línea traduciendo cada declaración en una secuencia de subrutinas y luego a código máquina. Esto hace que sea más flexible que

otros lenguajes de programación proporcionando un programa ejecutable más pequeño.

3.1.2. Es orientado a objetos

Python es un lenguaje de programación multiparadigma. Por esto, soporta el paradigma orientado a objetos, las reglas y técnicas de programación que encapsulan código dentro de los objetos. Además, todo el código escrito en el fuente de *Python* está en forma de objetos y clases.

3.1.3. Es portable

Python tiene la capacidad de correr en una gran cantidad de plataformas de hardware con la misma interfaz. Corre perfectamente en todos los sistemas operativos como Windows, Linux, UNIX, Amigo, MacOS, etc.

3.1.4. Es extensible

Python es tan versátil y flexible que permite a los programadores agregar módulos existentes, de bajo o alto nivel, o crear sus propios módulos para agregarlos al intérprete. Estos módulos y paquetes de herramientas permiten a los programadores desarrollar de una forma portable y multiplataforma, creando y personalizando sus programas, aplicaciones, o herramientas de software para ser más eficientes.

3.1.5. Es de scripting

Python puede ser utilizado como un lenguaje de *scripting*, tanto en *Batch* como *Interactive*. Además, al ser un lenguaje de *script*, permite un fácil acceso a otros programas. Se puede compilar en *bytecodes* para construir aplicaciones grandes.

3.1.6. Es web

Python ofrece una variedad de opciones para el desarrollo de aplicaciones *web*, y esto se debe a su escalabilidad. La biblioteca estándar de *Python* incorpora diferentes protocolos para el desarrollo *web*, como *HTML*, *XML*, *JSON* y procesamiento de *emails*. También provee las bases para *FTP*, *IMAP*, y otros protocolos de Internet.

Siendo tan flexible, provee un fácil uso de la interfaz *socket*, además de bibliotecas como *Requests* (un cliente *HTTP*), *Beatifulsoup* (*parser HTML*), *Feedparser* (*parser RSS/Atom*), entre otras. Además de las bibliotecas mencionadas, tiene varios *frameworks web* como *Django* y *Flask*.

3.2. Django

Django es un *framework* de desarrollo *web* de código abierto, que respeta el patrón de diseño *MVC* (Modelo Vista Controlador). Fue diseñado como un *framework* del lado del servidor, que opera con bases de datos relacionales. Sin embargo, a lo largo de los años se fue adaptando a las necesidades y puede operar con bases de datos no relacionales (a través de paquetes de terceros).

El contenido en los proyectos de *Django* trabaja en 3 grandes bloques: *urls*, *templates* y *apps*. Se definen de forma separada y se conectan para cumplir con la entrega de contenido, que es parte de los principios de arquitectura poco acoplada que promueve *Django*. Las *urls* definen puntos de entrada de acceso al contenido. Los *templates* definen los puntos de salida que dan forma al contenido final. En el medio, las *apps* sirven como *middleware* entre las *urls* y los *templates*, agregando contenido desde una base de datos o la interacción de un usuario. Para servir código estático, sólo se necesitan crear y configurar *urls* y *templates*. Para contenido dinámico, se necesitan crear y configurar *apps*.

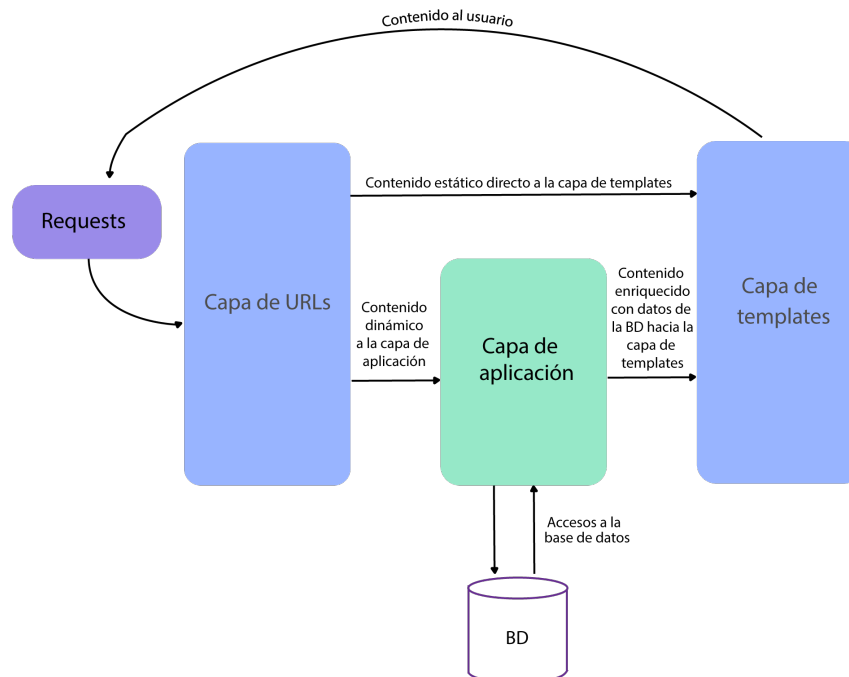


Figura 3.1: Flujo de trabajo de Django

La Figura 3.1 muestra dos flujos de información. Uno se utiliza para enviar información estática, y el otro se utiliza para enviar información dinámica.

3.2.1. ORM

Un *ORM* (*object-relational mapper*, o mapeo objeto-relacional) es una biblioteca que automatiza la transferencia de datos almacenados en tablas de una base de datos relacional en objetos.

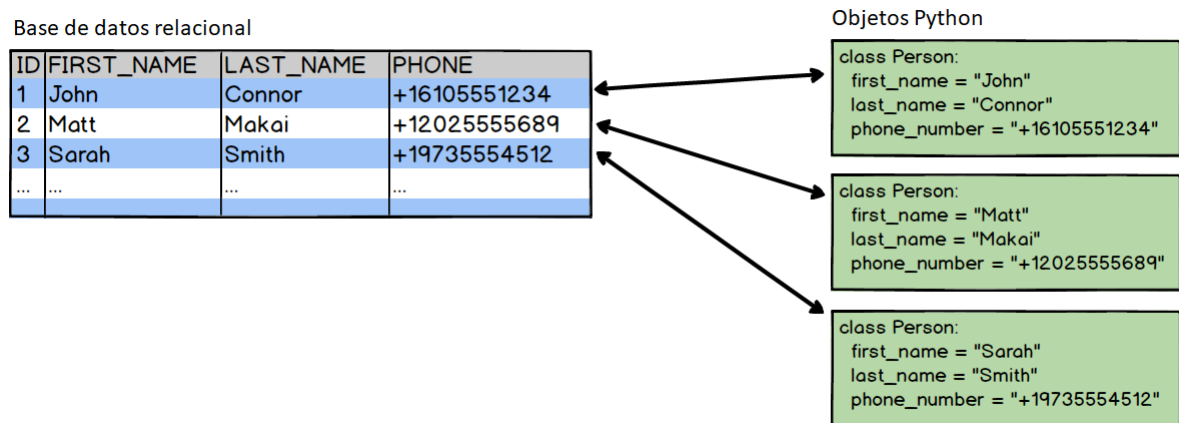


Figura 3.2: ORM

Los *ORMs* proveen un alto nivel de abstracción con respecto a bases de datos relacionales, permitiéndole al desarrollador escribir código *Python* en lugar de *SQL* para crear, leer, actualizar o borrar datos y esquemas en las bases de datos (Figura 3.2).

Por ejemplo, sin un *ORM* debería escribirse la siguiente declaración *SQL* para traer todas las filas en la tabla *USERS* de aquellos usuarios cuyo código postal sea 1234:

```
1 SELECT * FROM USERS WHERE codigo_postal=1234;
```

La *query* equivalente con *Django ORM* se escribe con el siguiente código *Python*

```
1 # Se obtienen todos los usuarios con codigo postal 1234
2 users = Users.objects.filter(codigo_postal=1234)
```

La posibilidad de poder escribir código *Python* en lugar de *SQL* puede acelerar el proceso de desarrollo, sobretodo al principio del proyecto, ya que no se tiene que cambiar entre código *Python* y declaraciones *SQL*. Si bien algunos desarrolladores no tienen problema en hacer este cambio de contexto, en general es más simple arrancar con un prototipo o una aplicación en un solo lenguaje.

Los *ORMs* además hacen posible intercambiar entre diferentes bases de datos relacionales. Por ejemplo, se podría arrancar a desarrollar con *SQLite* y luego cambiar a *Postgresql* o *mysql* en producción, haciendo cambios mínimos.

3.3. *Flask*

Flask es un *framework* pequeño con respecto a los estándares de muchos *frameworks web*, por eso se lo llama “*micro-framework*” [Fsc]. Pero esto no significa que sea menos que otros *frameworks*, sino que fue diseñado para ser extensible desde su creación. Provee un núcleo sólido con los servicios básicos, mientras el resto puede ser provisto por extensiones.

El hecho que se pueda elegir y poner la extensión que se desee, permite usar un *framework* que no tiene funcionalidades de más y sirve para exactamente lo que se necesite.

Flask tiene tres dependencias principales: rutas, *debugging* y *WSGI* (*Web Server Gateway Interface*, provisto por *Werkzeug*); los *templates* son provistos por *Jinja2*, y la interacción por línea de comandos viene de *Click*.

Flask no dispone de acceso nativo a bases de datos, ni valida formularios *web*, ni autentica ni realiza otras tareas de alto nivel. Sólo se pueden usar a través de extensiones. Como desarrollador, se puede decidir integrar la extensión que se desee o escribir una propia. Esto va en contraste con grandes *frameworks*, ya que es complejo o imposible cambiar estas funcionalidades.

3.4. *Python* en Ciencia de Datos

Python está ganando adeptos en la comunidad de la ciencia de datos, dominada históricamente por el lenguaje *R*, ya que provee a los científicos una gran cantidad de funcionalidades y les permite crear sus propias funcionalidades para realizar cálculos muy complejos. Además, les permite generar varios tipos de reportes de análisis, histogramas y grafos. Existe una gran variedad de módulos de Ciencia de Datos entre los que se destacan *NumPy* y *Pandas*.

3.4.1. *NumPy*

NumPy, abreviatura de *Numerical Python*, es una parte fundamental para la computación numérica. Provee estructuras de datos, algoritmos y bibliotecas necesarias para la mayoría de las aplicaciones científicas que involucran datos numéricos en *Python*. *NumPy* contiene entre otras cosas:

- *ndarray*: Un array multidimensional rápido y eficiente.
- Funciones para realizar cálculos basados en elementos con matrices u operaciones matemáticas entre matrices.
- Herramientas para leer y escribir *datasets* al disco.
- Operaciones de álgebra lineal y generación de números aleatorios.
- Una API en C muy madura que permite a las extensiones de *Python* acceder a las estructuras de datos de *NumPy*.

Más allá de las capacidades de procesamiento de matrices que *NumPy* le agrega a *Python*, uno de sus principales usos para análisis de datos es como contenedor para que los datos sean pasados entre algoritmos y bibliotecas. Para datos numéricos, los *arrays* de *NumPy* son más eficientes para guardar y manipular los datos que cualquier otra biblioteca de *Python*.

3.4.2. *Pandas*

Pandas provee estructuras de datos de alto nivel y funciones diseñadas para trabajar con datos estructurados o tabulados de forma rápida, fácil y expresiva. Desde su salida en 2010 contribuyó a que *Python* fuera un entorno de análisis de datos poderoso y productivo. Los objetos principales de *Pandas* son:

- *DataFrame*: una estructura de datos tabular, orientada a columnas, con etiquetas por columna y fila.
- *Series*: un array etiquetado de una dimensión.

Pandas mezcla la alta performance de las ideas computacionales sobre *arrays* de *NumPy*, con las capacidades flexibles de la manipulación de datos de las hojas de cálculos y las bases de datos relacionales (como *SQL*). Además, provee funcionalidades de indexación sofisticadas para que resulte fácil la remodelación, corte, agregación y selección de subconjunto de datos.

Pandas surgió en 2008 (previo a ser incorporado como una herramienta de *Python*) para cumplir con ciertos requerimientos que ninguna otra herramienta podía satisfacer:

- Estructuras de datos con ejes etiquetados, y que soporte alineación de datos automática o explícita. Esto previene errores comunes resultantes de trabajar con datos provenientes de distintas fuentes.
- Series de tiempo integradas.
- Estructura de datos que pueda manejar series de tiempo y otros datos a la vez.
- Operaciones aritméticas y reducciones que preserven los metadatos.
- Manejo de datos faltantes de forma flexible.
- Unión y otras operaciones relacionales encontradas en bases de datos.

3.5. *Javascript*

Javascript es el lenguaje predominante de programación de la *web*. Todos los navegadores modernos, consolas, *tablets*, *smartphones* incluyen un intérprete de *Javascript*, haciéndolo uno de los lenguajes más utilizados.

3.5.1. *React*

React es una biblioteca de *Javascript* que tiene como propósito simplificar el desarrollo de interfaces visuales. Fue desarrollado por Facebook y lanzado al mundo en 2013. Su objetivo principal es facilitar el razonamiento sobre las interfaces y su estado en cualquier momento, dividiendo la interfaz de usuario en una colección de componentes.

3.5.1.1. **Componente**

Los componentes permiten separar la interfaz de usuario en piezas independientes, reutilizables y diseñar cada pieza de software de forma aislada. Conceptualmente, los componentes son como las funciones de *Javascript*. Aceptan entradas arbitrarias (llamadas “props”) y devuelven a *React* elementos que describen lo que debe aparecer en la pantalla. Los componentes pueden referirse a otros componentes en su salida. Esto nos permite utilizar la misma abstracción de componente para cualquier nivel de detalle. Un botón, un cuadro de diálogo, un formulario, una pantalla en aplicaciones de *React*, se diseñan comúnmente como componentes.

3.5.1.2. DOM Virtual

DOM (*Document Object Model*) es un árbol que representa una página, empezando con la etiqueta `<html>`, bajando por cada hijo llamado *nodo*. Se guarda en la memoria del navegador y se vincula directamente con lo que se ve en una página. El *DOM* tiene una *API*, con la cual se puede acceder a él, acceder a cada nodo, filtrarlos, modificarlos. *React* mantiene una copia de la representación del *DOM* al que llama *DOM* virtual.

Cada vez que el *DOM* cambia, el navegador tiene que realizar dos operaciones intensivas: *repintar* (cambios visuales o de contenido en un elemento que no afectan el diseño y el posicionamiento en relación con otros elementos) y *reflujo* (recalcular el diseño de una parte de la página, o el diseño completo de la página). *React* usa *DOM* Virtual para ayudar al navegador a utilizar menos recursos cuando se necesita que haya cambios en la página.

React sólo cambia el *DOM* cuando el estado de un componente cambia explícitamente. Cuando hay un cambio, *React* actualiza el *DOM* Virtual relativo al componente que necesita cambiar. La clave está en que sólo actualiza el *DOM* una vez, así el *repintado* y *reflujo* que tiene que realizar el navegador se hace sólo una vez.

3.5.1.3. Flujo de datos unidireccional

El flujo de datos unidireccional (Figura 3.3), que no es único de *React*, es una forma en que los datos se transfieren a otras partes de la aplicación en un único sentido. Esto en *React* significa que:

- El estado es pasado a la vista y a los componentes hijos.
- Las acciones son desencadenadas por la vista.
- Las acciones pueden actualizar el estado de un componente.
- El cambio de estado es pasado a la vista y a los componentes hijos.

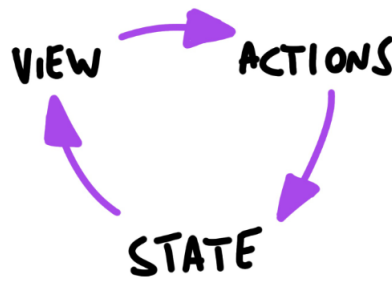


Figura 3.3: Flujo de datos unidireccional

Gracias a este único sentido, los datos no pueden fluir hacia el lado contrario. Esto trae algunas ventajas:

- Es menos propenso a los errores, ya que se tiene más control sobre los datos.
- Es mas fácil de depurar los errores, ya que se sabe *qué* viene y desde *dónde*.
- Es más eficiente, ya que la biblioteca ya sabe cuáles son los límites de cada parte de la aplicación.

Un estado siempre le pertenece a un componente. Cualquier dato que es afectado por ese estado, sólo puede afectar a los componentes que estén por debajo: sus hijos. Cambiar el estado de un componente nunca afecta a los padres o sus hermanos. Por esta razón, muchas veces el estado se lleva a componentes más arriba en el árbol de componentes, así puede ser compartido por otros componentes que lo necesiten.

3.5.1.4. **Axios**

Axios es una biblioteca de *Javascript* muy popular que se usa para realizar *Requests HTTP* y que funciona tanto en navegadores como en plataformas *Node.js*. Tiene soporte para todos los navegadores modernos, incluido IE8 y superiores. Está basado en *promises* y nos permite escribir código asíncronico para realizar *Requests XHR* de manera amigable. Usar *Axios* tiene algunas ventajas por sobre la *API* nativa *Fetch*:

- Tiene soporte para navegadores antiguos.
- Permite cancelar un *request*.
- Permite setear un tiempo límite para la respuesta del *request*.

- Tiene protección para *CSRF* incluida.
- Tiene soporte para progresos de carga.
- Realiza una transformación automática de datos a *JSON*.
- Funciona en *Node.js*.

3.6. Microservicios

Cuando se programan aplicaciones monolíticas (todas las funcionalidades en una sola aplicación), a medida que el código va creciendo cuando se agregan funcionalidades, se hace cada vez complejo saber dónde deben implementarse los cambios. Las funcionalidades nuevas empiezan a quedar distribuidas por el código, dificultando la tarea de encontrar errores.

Los microservicios son servicios pequeños y autónomos que trabajan en conjunto para afrontar un acercamiento hacia la independencia de funcionalidades, enfocándose en la obiedad donde el código sólo sirve para alguna funcionalidad. Y al mantener este servicio enfocado dentro de sus límites, se evita que crezca demasiado.

3.6.1. Heterogeneidad Tecnológica

Con un sistema compuesto de múltiples servicios colaborando entre sí, se puede elegir usar diferentes tecnologías dentro de cada uno de éstos. Esto permite elegir la herramienta correcta para cada trabajo, en lugar de tener que elegir una opción que sirva un poco para todo, y puede terminar siendo perjudicial para el sistema.

Si una parte del sistema necesita mejorar su performance, se puede decidir cambiar de tecnología a una que resulte mejor para esa tarea en particular.

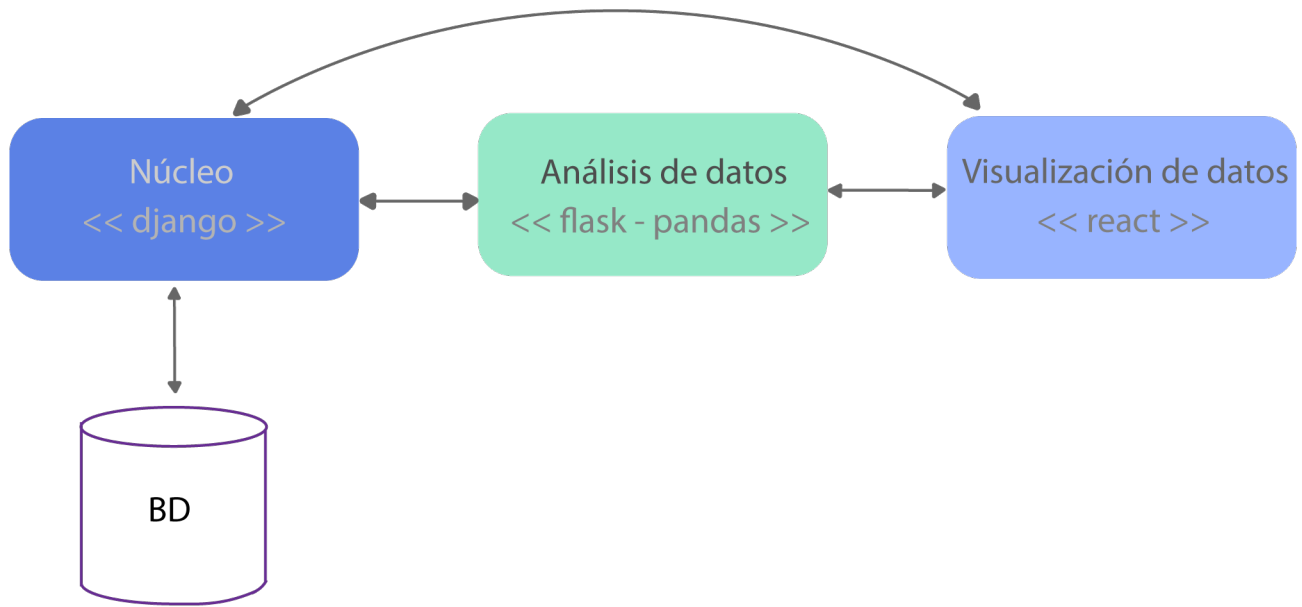


Figura 3.4: Microservicios: Homogeneidad Tecnológica

El diseño de la aplicación de *Seguimiento Académico* que se desarrolla en este trabajo sigue el diseño de la Figura 3.4, que muestra tres servicios distintos, un núcleo que usa *Django*, un servicio de análisis de datos, con *Flask* y *Pandas*, y un servicio de visualización de datos con *React*.

3.6.2. Independencia

En un sistema monolítico, cuando hay una falla todo el sistema se corrompe. Aunque se pueda mitigar usando varias instancias del mismo sistema, esto no es conveniente ya que a medida que el sistema va creciendo, tener varias instancias de un sistema grande necesitaría una mayor capacidad de procesamiento. Desde la perspectiva de microservicios, cuando uno de los servicios tenga una falla y esa falla no genere un problema en cascada, se puede aislar el problema y así el resto de los servicios puede seguir funcionando.

3.6.3. Escalamiento

En un sistema monolítico, todo se escala junto. Si sólo se necesitara que un servicio determinado tenga más recursos, estaría dándole más recursos a todo el sistema. En cambio, con microservicios, si en un determinado momento se necesita, por ejemplo, que el servicio de análisis de datos procese más peticiones se podría escalar sólo ese servicio teniendo dos instancias (o más) de esa parte del sistema, dejando los otros módulos corriendo con hardware menos poderoso,

acorde a sus necesidades.

La Figura 3.5 muestra cómo podría ser escalado el sistema en caso de ser necesario. El Núcleo muestra 4 instancias, el análisis dos y la visualización una sola instancia, a modo de ejemplo.

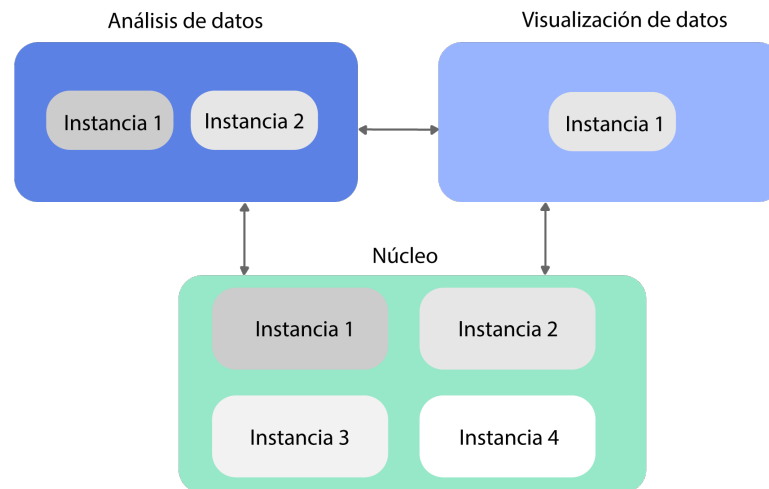


Figura 3.5: Microservicios: escalamiento

3.6.4. Facilitar *deploy*

Hacer un pequeño cambio en un sistema monolítico requeriría *deploy* toda la aplicación para hacer efectivo el cambio. Esto genera un gran impacto y un alto riesgo. Desde la perspectiva de los microservicios, el cambio que se hace sobre un servicio permite *deploy* sólo ese módulo, sin que los otros sepan que hubo una modificación. Además, el código es desplegado más rápido. De haber un problema en la actualización, se puede aislar a un sólo servicio, permitiendo retroceder a una versión anterior.

3.6.5. Reemplazabilidad

El costo de reemplazar un servicio por una mejor implementación es más fácil de manejar cuando hay independencia. Si en un futuro se quisiera, por ejemplo, cambiar el módulo de encuestas, sólo haría falta reemplazar ese pequeño sistema sin tener que modificar el resto. Ni siquiera deberían enterarse del cambio.

3.7. REST

REpresentational State Transfer (Transferencia de estado representacional, su traducción) es un estilo arquitectural inspirado en la Web.

Una parte importante es el concepto de *recursos*. El servidor crea diferentes representaciones del recurso en un *request* y la forma en que es enviado está desacoplado a cómo está guardado internamente. Cuando un cliente pide por un recurso, se le da una representación con la estructura de *JSON*.

A lo largo de los años, se desarrollaron muchos servicios basados en la arquitectura *REST*, que usa las funcionalidades provistas por la capa de aplicación del protocolo *HTTP* [Res] [New12]. Esto resultó en un incremento en el interés comparado al tradicional *SOAP* (*Simple Object Access Protocol*). Además, grandes compañías como Twitter o Amazon usan interfaces del tipo *REST* en sus servicios, lo cual se puede ver en la documentación de sus *APIs* (*Application Programming Interface*). Mas allá de la tendencia, no hay estándares o guías de cómo desarrollar un servicio *web restful*. Solamente existen buenas prácticas.

3.7.1. Descripción de los recursos

La descripción de los recursos se relaciona con la usabilidad de los servicios *web*, ya que éstos son una representación del modelo. Para una correcta descripción, existen algunas buenas prácticas [WAP]:

- Deberían usarse sustantivos para los recursos.
- El nombre del recurso debería ser un nombre específico del dominio, para que la semántica pueda ser interpretada por cualquier usuario sin conocimientos adicionales.
- Debería evitarse la mezcla del uso del plural y singular para garantizar coherencia.
- Debería usarse la convención de nombres de *Javascript*, ya que el tipo *JavaScript Object Notation (JSON)* es el tipo de dato más usado para la comunicación entre el cliente y el servidor.

3.7.2. Identificación de recursos

Deberían usarse *URIs* para la identificación única de recursos [Fie00]. Para esto existen algunas buenas prácticas [WAP]:

- Una *URI* debería ser autoexplicativa, de forma tal que no debería necesitar información adicional para su uso.

- No deberían existir verbos en la *URI*, ya que esto implica una orientación a métodos como *SOAP*.
- Un recurso debería ser representado por dos *URIs*. La primera para representar el conjunto de estados de un recurso específico, y la segunda para representar un estado en particular de ese conjunto de estados.
- El identificador de un estado específico debería ser difícil de predecir y no referenciar objetos directamente, según *OWASP (Open Web Application Security Project)*, si no existiera una capa de seguridad.

3.7.3. Manejo de errores

Los mensajes de error deben ser claros y entendibles, para que su causa pueda ser fácilmente identificada. Con este requerimiento se pueden reconocer algunas buenas prácticas [[HTT](#), [WAP](#)]:

- La cantidad de códigos de estados de *HTTP* debería estar limitado para reducir el esfuerzo de buscar en la especificación.
- El uso de los códigos de estado de *HTTP* tienen que corresponderse con la especificación oficial de *HTTP*.
- Debería darse un mensaje de error detallado como una pista del error causado del lado del cliente. Por este motivo, el mensaje de error debería tener los siguientes ingredientes:
 - Un mensaje para los desarrolladores, que describe la causa del error y algunas pistas sobre cómo resolver el problema.
 - Un mensaje que puede ser mostrado al usuario
 - Un código de error específico de la aplicación
 - Un link para más información sobre el problema.

3.7.4. Uso de parámetros

Cada *URI* de un recurso puede extenderse con parámetros para proveer información opcional al servicio. A continuación se detallan cuatro casos de uso:

- *Filtrado*: para filtrar información de un recurso, puede usarse sus atributos o algún lenguaje de *queries*. La elección de una de estas variantes, depende de la necesidad del poder de expresión que se tenga para filtrar.

- *Ordenamiento*: para ordenar la información, se recomienda [BPR] una lista de atributos separados por coma con el parámetro *sort* en la *URI*, seguido por un signo de más (+) como prefijo para un orden ascendente, o un signo menos (-) para un orden descendente.
- *Selección*: la selección de información en forma de atributos reduce el tamaño de transmisión sobre la red, respondiendo sólo con la información pedida. Para este propósito, se recomienda una lista de atributos separadas por coma.
- *Paginación*: La paginación permite partir la información en varias páginas virtuales, mientras se referencian la página anterior, la próxima, la primera y la última.

3.7.5. Interacción con los recursos

Usando *REST* como el estilo de arquitectura subyacente de un sistema, el cliente interactúa con las representaciones de un recurso en lugar de usarlo directamente. La interacción entre el cliente y el servidor está construido en la capa de aplicación del protocolo *HTTP*, que ya provee cierta funcionalidad para la comunicación. Para la interacción con un recurso, se identifican tres buenas prácticas diferentes:

- El uso de los métodos *HTTP* deberían ser de acuerdo a las semánticas definidas por la especificación oficial de *HTTP* [WAP]. Así, el método *GET* de *HTTP* sólo debería usarse para las operaciones sin efectos secundarios. Para una mejor visión general, la Tabla 3.1 muestra los métodos *HTTP* más usados y sus características. Estas características pueden ser usadas para asociar los métodos *HTTP* con el correcto uso de la creación, lectura, edición y borrado de recursos (*CRUD*) [Vin08].
- El soporte de la operación *OPTIONS* es recomendada si una gran cantidad de datos tienen que transmitirse, ya que permite al cliente pedir los métodos soportados de la representación actual antes de transmitir la información por un medio compartido.
- El soporte del *GET* condicional debería ser considerado durante el desarrollo de un servicio basado en *HTTP* ya que previene al servidor de transmitir datos ya enviados anteriormente. Sólo si hay modificaciones de la información solicitada desde la última solicitud, el servidor responde con la última representación [Vin08].

Método	Seguro	Idempotente
POST	No	No
GET	Si	Si
PUT	No	Si
DELETE	No	Si

Tabla 3.1: Características de los métodos HTTP más comunes

3.7.6. *REST* y *HTTP*

HTTP define algunas funcionalidades que son muy útiles para *REST*. Por ejemplo los verbos (GET, POST, PUT, etc) ya tienen un buen entendimiento sobre cómo deberían funcionar con recursos. El estilo de arquitectura *REST* nos dice que los métodos deberían comportarse de la misma forma en todos los recursos, y la especificación *HTTP* define muchos de estos métodos. GET sirve para pedir recursos y POST para crear (aunque para el sistema seguimiento académico de no todos los recursos van a poder ser creados). *HTTP* también trae un gran ecosistema de herramientas y tecnologías que facilitan la tarea.

3.8. JSON Web Tokens (JWT)

3.8.1. ¿Qué es?

JSON Web Token (*JWT*) es un estándar abierto (RFC 7519) que define una forma compacta de transmitir información entre partes de forma segura como un objeto *JSON*. Esta información puede verificarse porque está firmada digitalmente. *JWT* puede ser firmado usando una clave secreta (con el algoritmo *HMAC* [*HMA*]) o un par de claves pública/privada usando *RSA* [*RSA*] o *ECDSA* [*ECD*].

3.8.2. ¿Cuándo usarlo?

Estos son algunos escenarios donde *JWT* es útil:

- *Autorización*: Este es el escenario más común. Una vez que el usuario está *logueado*, cada *request* debe incluir el *JWT*, permitiendo al usuario acceder a rutas, servicios y recursos que le son permitidos con ese *token*. *Single Sign On* es una funcionalidad que usa mucho *JWT*, por su habilidad de poder ser usado a través de diferentes dominios.
- *Intercambio de información*: *JSON Web Tokens* son una forma segura de transmitir información entre partes. Como están firmados, se puede estar

seguro que el emisor es quien dice qué es. Además, como la firma es calculada usando el *header* y el *payload*, se puede verificar que el contenido no fue modificado.

3.8.3. Estructura

En su forma compacta, los *JSON Web Tokens* consisten de tres partes separadas por puntos (.), que son:

- *header*
- *payload*
- Firma

3.8.4. *header*

El *header* consiste de dos partes: el tipo de *token*, que es *JWT*, y el algoritmo que se usa para firmar, como *HMAC SHA256* [[SHA](#)].

```
1 {  
2   "typ": "JWT",  
3   "alg": "HS256"  
4 }
```

Luego, este *JSON* es encodeado con *Base64Url* [[BAS](#)] para formar la primera parte del *JWT*.

3.8.5. *payload*

La segunda parte del token es el *payload* que contiene los pedidos. Un pedido es una declaración sobre una entidad (por lo general, un usuario) e información adicional. Hay tres tipos de pedidos: registrados, públicos y privados.

- *Registrados*: son un conjunto de pedidos predefinidos, que no son obligatorios pero recomendados para proveer información útil. Para el presente trabajo, solamente se usa *exp* (*fecha de expiración del token*).
- *Públicos*: Esta parte es definida por quien use el *JWT*. Pero para evitar colisiones, deberían estar definidos en el registro *IANA* o estar definidos como una *URI* que contiene un nombre resistente a colisión.

- *Privados*: Estos son campos personalizados, con la finalidad de compartir información entre partes.

Un ejemplo de *payload* puede ser:

```

1  {
2    "token_type": "refresh",
3    "exp": 1582059853,
4    "jti": "e9b85778f3f44decba69a611e4e1c700",
5    "user_id": 1
6  }
```

El *payload* luego es encodeado con *Base64Url* para formar la segunda parte del *JWT*.

3.8.6. Firma

Para crear la parte de la firma, se usa el *header encodeado*, el *payload encodeado*, una clave secreta, y el algoritmo especificado en el *header*. Por ejemplo, usando el algoritmo *HMAC SHA256*, la firma se crea de la siguiente forma:

```

HMACSHA256(
    base64UrlEncode(header) + "." +
    base64UrlEncode(payload) ,
    clave-secreta
)
```

La firma se usa para verificar que el mensaje no fue cambiado en el camino, y también para verificar que el emisor es quien dice ser.

3.8.7. “Ensamblando las diferentes partes”

El resultado van a ser tres *strings* en *Base64Url*, separados por puntos que pueden ser fácilmente pasados por *HTTP*, siendo muy compactos comparado con estándares *XML*.

El cuadro siguiente muestra un *JWT* formado con las tres partes provistas:

```

bGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
b2t1bl90eXB1IjoicmVmcmVzaCI6ImV4cCI6MTU0MjA1OTg1MywianRpIjoizTli
NzhmM2Y0NGRlY2JhNjlnNjExZTRlMWM3MDAiLCJ1c2VyX2lkIjozLCJjYXJyZXJh
WyJXIl0sInVzZXJ1YW1lIjoieWRtaW4ifQ.
oqAQpHNliUUw3-xLJE2u4M6_tBByJQt7IFYQis
```

3.8.8. ¿Cómo funciona?

En autenticación, cuando el usuario accede usando sus credenciales, se le provee un *JSON Web Token*. Como es una credencial, se debe tener cuidado con los problemas de seguridad que se puedan tener. En general, no se deben mantener *tokens* más tiempo del requerido. Tampoco deben guardarse datos sensibles de sesiones en el almacenamiento del navegador. Cuando un usuario quiere ingresar a una ruta o un recurso protegido, tiene que mandar el *JWT* en el *header* de autorización, usando el esquema “*Bearer*”. El contenido del *header* tiene que verse como se muestra a continuación:

```
Authorization: Bearer <token>
```

Este puede ser un mecanismo de autorización sin estado. Las rutas protegidas del servidor chequean que el *JWT* sea válido. Si lo es, el usuario puede ingresar a esas rutas protegidas. Si el *JWT* tiene los datos necesarios, se reducen las *queries* a la base de datos, aunque no siempre sea el caso. Si el *token* es enviado en el *header* de autorización, *Cross-Origin Resource Sharing (CORS)* [COR] no va a ser un problema ya que no usa *cookies*.

3.8.9. Problemas que resuelve

A pesar de que el principal propósito de *JWTs* es transferir *Requests* entre dos partes, el aspecto más importante es el de estandarizar una estructura de datos de forma simple y encriptada. Los principales problemas que resuelve son:

- Autenticación
- Autorización
- Identidad Federada
- Sesiones del lado del cliente

3.8.10. Client-side/Stateless Sessions

Las sesiones sin estado (*stateless sessions*) son en realidad datos del lado del cliente (*client-side*). El aspecto fundamental de esta aplicación se basa en el uso de firmas y posiblemente encriptación para proteger el contenido de la sesión.

Como los datos del lado del cliente pueden ser manipulados con facilidad, se tiene que tener mucho cuidado desde el *backend*.

JWTs, en virtud de *JSON Web Signature (JWS)* [JWS] y *JSON Web Encryption (JWE)* [JWE], puede proveer distintos tipos de firmas y encriptación. Las firmas

son útiles para validar los datos contra posibles manipulaciones. La encriptación es útil para proteger los datos de ser leídos por terceros.

3.8.11. ¿Es útil tener una sesión del lado del cliente?

Existen pros y contras a cualquier decisión, y las sesiones *client-side* no son la excepción. Algunas aplicaciones pueden requerir sesiones muy grandes. Enviando este estado ida y vuelta hacia el *backend* por cada *request* (o grupo de *requests*) puede perder rápidamente los beneficios que trae *JWT*. Es necesario un buen balance entre los datos del lado del cliente y las búsquedas a la base de datos del *backend*.

3.8.12. Tokens de acceso y *refresh*

Los *tokens* de acceso (*access*) y *refresh* son dos tipos de *tokens* que ayudan en el contexto de autenticación y autorización.

Los *tokens* de acceso son *tokens* que dan acceso a recursos protegidos. Estos *tokens* son de corta vida y tienen una fecha de expiración como dato. Tienen, además, otra información que puede ser de ayuda para identificar al cliente y está definida en la implementación.

Por el contrario, los *tokens* de *refresh* le dan permiso a los clientes que pidan un nuevo acceso. Por ejemplo, luego de que un *token* de acceso expiró, un cliente puede hacer un pedido de un nuevo acceso al servidor de autorización. Para que esto pueda suceder, es requerido un *token* de *refresh*. A diferencia de los *tokens* de acceso, el tiempo de vida del *token* de *refresh* suele ser largo.

La principal diferencia entre el *token* de acceso y el de *refresh*, está en la posibilidad de hacer los *tokens* de acceso fáciles de validar. Un *token* de acceso que tiene una firma, no hace falta que sea validado por un servidor de autorización. Los *tokens* de *refresh*, por el contrario, necesitan que se acceda al servidor de autorizaciones. Manteniendo la validación separada de las *queries* al servidor de autorización, es posible obtener una mejor latencia. Para asegurar que la pérdida o robo de *tokens* no sea determinante, se debería poner un tiempo de vida corto. Los *tokens* de *refresh* al ser de vida prolongada, tienen que ser protegidos contra estas incidencias. Una opción es agregarlo a una lista negra y cuando alguien pida un *access token*, éste expira automáticamente.

3.9. Docker

3.9.1. Introducción

Docker es una plataforma abierta para desarrollar, transportar, y ejecutar aplicaciones de forma rápida. *Docker* permite que las aplicaciones corran de forma separada de la infraestructura del *host*. Permite también enviar código, testear, desplegar rápido y acortar el ciclo entre escribir el código y correrlo. *Docker* logra esta funcionalidad combinando una plataforma de virtualización liviana con herramientas que permiten manejar y desplegar aplicaciones [Dj].

Docker usa funcionalidades del kernel de Linux como *cgroups*, que limita y aísla el uso de recursos, y *namespaces*, que permite a los contenedores independientes correr en una única instancia de Linux evitando la sobrecarga de iniciar máquinas virtuales. Además, provee una forma para correr aplicaciones aisladas en un contenedor de forma segura, y que estos contenedores puedan correr de forma simultánea en un mismo *host* compartiendo el mismo *kernel*. Aunque cada contenedor puede usar una cantidad de recursos definidos por el *host*.

A diferencia de las máquinas virtuales, no se requiere un sistema operativo separado, sino que depende de las funcionalidades del *kernel* y el uso aislado de recursos (CPU, memoria, I/O, red, etc).

3.9.2. Imágenes y Contenedores

Un contenedor es una versión de un sistema operativo Linux, sólo con los componentes más básicos. Una imagen es un software que se carga dentro del contenedor al momento de ejecutar el comando `run`.

```
docker run hello-world
```

El comando `run` recibe como parámetro requerido el nombre de la imagen que se desea cargar en un contenedor, en este caso, `hello-world`. Al correr dicho comando, *Docker* ejecuta las siguientes acciones:

- Se comprueba que exista en el sistema una imagen con el nombre `hello-world`.
- En caso que no exista, se descarga desde el repositorio de imágenes configurado (por defecto es *Docker Hub*).
- Se carga la imagen en el contenedor y se ejecuta.

Por otro lado, una imagen de *Docker* puede ejecutar desde un simple comando hasta cargar un complejo sistema de base de datos. Para construir una imagen, es necesario crear un archivo llamado `Dockerfile`.

```
FROM ubuntu:16.04  
RUN apt-get -y update  
CMD["echo Hola"]
```

El Dockerfile anterior busca una imagen de Ubuntu con el tag 16.04. Luego ejecutará un comando para actualizar los paquetes del sistema operativo y finalmente mostrará el mensaje Hola. El comando para construir una imagen de *Docker* es:

```
docker build -t nombre-imagen
```

Se ejecutará el comando build para construir la imagen. El argumento -t indica que se pondrá la etiqueta nombre-imagen a la imagen. El punto final indica el directorio de contexto de la imagen. En este caso, el contexto será el directorio donde se encuentra el Dockerfile. Luego se carga la imagen en un contenedor con el comando:

```
docker run nombre-imagen
```

3.9.3. Contenedor

Los contenedores y las máquinas virtuales tienen un aislamiento y asignación de recursos similar, aunque funcionan diferente ya que los contenedores virtualizan el sistema operativo en lugar del hardware.

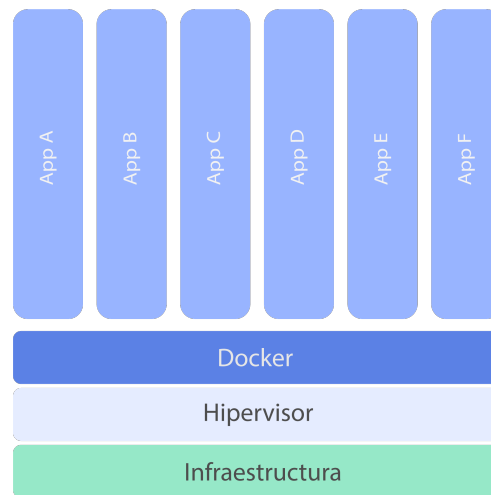


Figura 3.6: Estructura de contenedores

Los contenedores son una abstracción de la capa de aplicación que empaqueta el código y las dependencias juntos. Múltiples contenedores pueden correr en la misma máquina compartiendo el *kernel* del sistema operativo junto con otros contenedores, todos corriendo de forma aislada (Figura 3.6). Además, los contenedores ocupan menos espacio que las VMs.

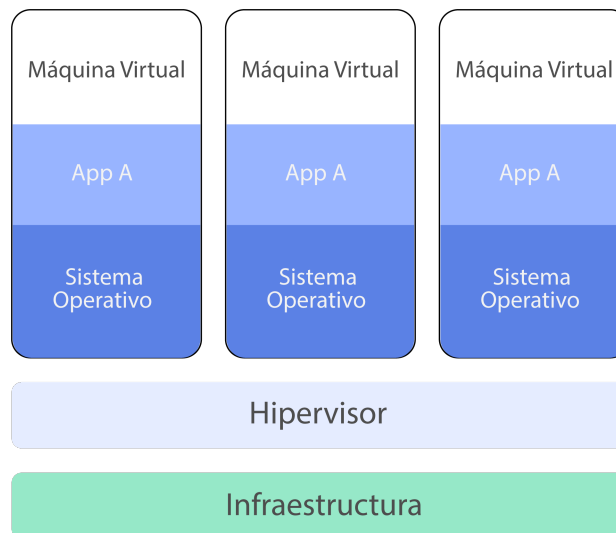


Figura 3.7: Estructura de Máquinas virtuales

Las máquinas virtuales (VMs) son una abstracción del hardware físico, transformando un servidor en múltiples servidores. El *hypervisor* permite que múltiples VMs corran en una misma máquina (como muestra la Figura 3.7). Cada VM incluye una copia de un sistema operativo, la aplicación, los binarios y bibliotecas, ocupando decenas de GBs.

3.9.4. *Docker Compose*

Docker Compose es una herramienta que permite correr un sistema formado por múltiples contenedores. Para ello, se debe crear un archivo *.yml* en el que se definan los servicios con los que va a contar la aplicación. Cada servicio estará formado por un contenedor corriendo una imagen de *Docker*. Para cada servicio pueden definirse nombres, puertos expuestos, conexiones de red, etc.

3.9.5. *Docker Swarm*

Las funcionalidades de administración y orquestación incluidas en *Docker Engine* fueron desarrolladas usando *swarmkit*. *Swarmkit* es un proyecto independiente que implementa la capa de orquestación de *Docker*. Un *swarm* consiste en múltiples *hosts* de *Docker* que corren en modo *swarm* y actúan como *managers* (administrando membresías y delegaciones) y *workers* (que corren servicios *swarm*). Un *host* de *Docker* puede ser *manager*, *worker* o realizar ambas tareas. Al momento de crear un servicio, se define un estado óptimo (número de réplicas, redes, almacenamiento asignado y puertos expuestos para los servicios). *Docker* trabaja para mantener ese estado deseado. Por ejemplo, si un nodo se encuentra no disponible, *Docker* reasigna las tareas de ese nodo a otros nodos. Una tarea es un contenedor que es parte de un servicio *swarm* y es administrado por un *swarm manager*, a diferencia de un contenedor independiente.

Una de las ventajas principales de los servicios *swarm* por sobre los contenedores independientes, es que se pueden modificar las configuraciones del servicio (ya sea redes o volúmenes asignados) sin necesidad de reiniciar el servicio manualmente. *Docker* actualiza esa configuración, detiene el servicio con la configuración anterior, y crea un nuevo servicio con la configuración deseada.

Cuando *Docker* corre en modo *swarm*, también pueden correrse contenedores independientes en cualquiera de los *hosts* que participan del *swarm*. De la misma forma que se puede utilizar *Docker Compose* para definir y correr contenedores, se puede definir y correr *stacks* de servicios *Swarm*.

El modo *Swarm* tiene un componente de *DNS* interno que automáticamente asigna a cada servicio una entrada *DNS* dentro del *swarm*. El *swarm manager* usa un balanceo de carga interno que distribuye los pedidos entre los servicios basándose en el nombre *DNS* del servicio.

3.9.5.1. *Portainer*

Portainer es conjunto de herramientas gratuitas y de código abierto, destinadas al manejo y administración de contenedores. Funciona tanto para *Docker*, *Docker Swarm*, *Kubernetes*, etc.

Portainer remueve la complejidad asociada a la orquestación de contenedores, para que los desarrolladores puedan hacer un *deploy* de aplicaciones más rápido, fácil y de forma correcta. Además, puede utilizarse para observar el comportamiento de los contenedores, administrar las plataformas subyacentes, entre otras funcionalidades.

3.10. *Nginx*

Nginx es uno de los *web servers* más populares. Sirve tráfico *HTTP* y *HTTPs*, *proxys* a *Python*, *Node.js*, corre software como balanceador de carga, *http cache* y *SSL* (que son las funcionalidades relevantes al presente trabajo).

Nginx es una pieza de software extremadamente modular. Muchas de las funcionalidades que trae por defecto, son en realidad módulos que se pueden activar y desactivar en cualquier momento. Una de las ventajas es que se puede decidir qué módulos se quiere utilizar, y cuáles no. Una de las funcionalidades más importantes para el desarrollo de microservicios es el *reverse proxy*.

3.10.1. Forward proxy vs reverse proxy

Una de las razones más populares para usar *Nginx* es para que sirva aplicaciones dinámicas escritas en *Python* y *Node.js*.

A diferencia de *Apache*, *Nginx* no tiene la habilidad de embeber un intérprete al *webserver*, como hace *Apache* con *mod_php*. En su lugar, *Nginx* toma un enfoque mucho más liviano. Es simplemente un *webserver* y su tarea principal es correr una aplicación *web* delegándolo a un *server* y *proxies* separados.

3.10.2. Forward Proxy

Se le llama *forward proxy* a la configuración por defecto que se encuentra en todas las conexiones salientes de Internet. La Figura 3.8 muestra la estructura de funcionamiento.

Las conexiones salientes de una computadora son capturadas por el *forward proxy* y reenviadas a Internet. Para Internet, todas las computadoras aparecen como si vinieran del mismo lugar- el *forward proxy*.

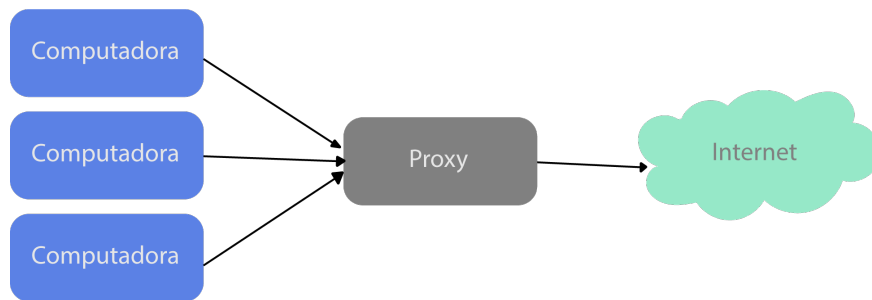


Figura 3.8: Funcionamiento de Forward Proxy

3.10.3. Reverse Proxy

Se le llama *reverse proxy* a lo opuesto de *reverse proxy*, y es una configuración muy común para servir aplicaciones web dinámicas y para balance de carga.

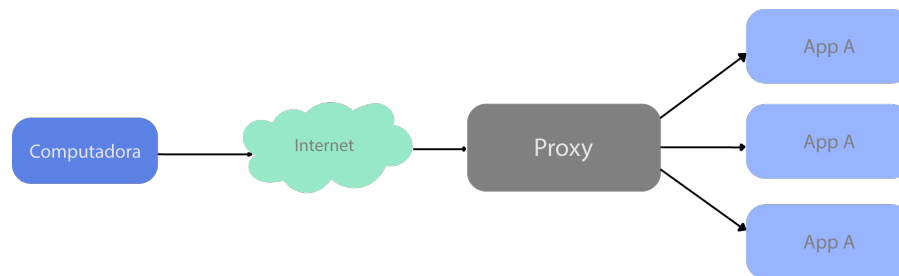


Figura 3.9: Funcionamiento de Reverse Proxy

En el ejemplo de la Figura 3.9, el *reverse proxy* hace de multiplexor para diversas conexiones de Internet, a una aplicación dinámica. El *reverse proxy* mira el *request*, y lo reenvía a la aplicación.

3.10.4. Balance de carga

Una de las formas más comunes de escalar una aplicación web es usar *balance de carga*. La idea detrás de esto, es distribuir la carga, o tráfico web, a través de diferentes *application servers*. El funcionamiento normal de una sola aplicación con *Nginx* es que los visitantes ingresan el nombre de dominio y son ruteados directamente al servidor donde *Nginx* está sirviendo la única aplicación. El problema es que cuando se tienen miles de conexiones visitando la aplicación, es probable que un solo servidor no pueda manejarlo. En este contexto es cuando juega un papel importante el balanceador de carga, que recibe todo el tráfico ingresante y lo distribuye a través de un conjunto de servidores de aplicación. Existen distintos tipos de balanceos de carga: por software y por

hardware. *Nginx* tiene la habilidad de correr como un balanceador de carga de software, usando el módulo `http_proxy`.

El balance de carga [Cor15] es en definitiva un *reverse proxy*, con algunas diferencias fundamentales:

- Los balanceadores de carga reenvían el tráfico a través de varios *backends*, mientras que un *reverse proxy* tradicional lo hace hacia uno solo.
- Los balanceadores de carga usualmente operan en la capa 7 (*HTTP*) o en la capa 4 (*TCP*) del modelo OSI [Tan02], cuando típicamente sólo operaríamos en la capa 7 usando un *reverse proxy* de aplicaciones modernas.
- La escala es crítica para los balanceadores de carga: en sitios ocupados, pueden ver entre 50 y 100 veces la cantidad de tráfico que recibe un servidor de aplicación. Por ejemplo, un servidor de aplicaciones potente sólo necesitará manejar entre 100 y 200 solicitudes por segundo, mientras que un balanceador de carga puede recibir más de 10,000 solicitudes por segundo.

3.11. Conclusiones

A partir de la elección del contexto tecnológico para la implementación de la aplicación de *Seguimiento Académico*, en este Capítulo se dieron las especificaciones técnicas de cada una de las tecnologías involucradas en el proceso. Como se dijo al principio del presente Capítulo, se decidió que la mejor solución era desarrollar tres aplicaciones diferentes. Por un lado, un núcleo que se encargase de manejar los datos y servirlos mediante una *API REST*. Esta decisión técnica es beneficiosa para el seguimiento académico pero también para que los datos puedan ser consultados por aplicaciones futuras.

Luego, para el análisis y visualización de los datos se decidió separarlo en dos aplicaciones diferentes. La primera, desarrollada con *Flask*, que conjuntamente con *Pandas* y *NumPy* se encargará de analizar y servir los datos analizados. La segunda, desarrollada con *React*, tendrá la tarea de mostrar los datos analizados. Esta decisión técnica se basa en desacoplar los módulos de tal manera que sean reemplazables en futuras mejoras del desarrollo presentado, o bien que puedan ser escaladas en base a necesidades propias de la aplicación.

Los detalles de la implementación de estas tres aplicaciones se detalla en el capítulo siguiente.

Capítulo 4

Desarrollo de la Solución

En el presente Capítulo, se explicará la implementación de la aplicación presentada en este informe.

4.1. Introducción

Las funcionalidades identificadas en los requerimientos del Capítulo 2 se implementaron en una aplicación formada por 3 aplicaciones a la vez, usando la arquitectura y tecnologías descritas en el Capítulo 3.

La aplicación "*Seguimiento Académico*" se conecta a una aplicación independiente que se encarga del análisis de los datos. Estos datos se obtienen gracias a una tercera aplicación independiente llamada "*Núcleo Académico*".

La Figura 4.1 muestra el funcionamiento de estas tres aplicaciones.

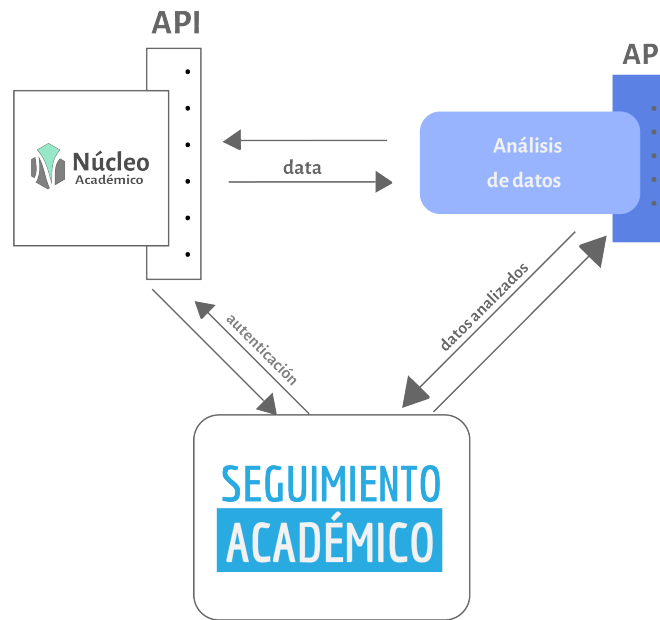


Figura 4.1: Obtención de datos a través del núcleo

Para el proceso de desarrollo de las aplicaciones se utilizó *GIT* como control de versiones del código y *GitHub* como repositorio del mismo. Además de usar *GitHub* como repositorio de código, se utilizó la herramienta de *issues* para poder tener una mejor trazabilidad del código.

Para el almacenamiento de imágenes de *Docker* se utilizó el repositorio *Docker Hub*.

Adicionalmente se utilizó *Travis* para tener un entorno de integración continua. Esto significa que a medida que el desarrollo iba creciendo, esta herramienta se encargó de *buildear* el código y correr los tests.

4.2. Núcleo

El núcleo se encarga de administrar los datos no sensibles de los estudiantes relacionados al desarrollo de la carrera en la cual se inscribieron. El núcleo tiene la capacidad de servir dichos datos para que sean consumidos por los usuarios que tengan los permisos correspondientes, según la carrera a la que pertenecen.

4.2.1. Tecnologías

El Núcleo fue desarrollado con *Python 3.6*, usando el *framework web Django* en su versión 3.0. *Django* incluye un administrador (*django-admin*) que genera automáticamente los listados, la creación, la edición y el borrado de los modelos



Figura 4.2: Logo del Núcleo

desarrollados. Además, trae un sistema de autenticación de usuarios que facilita la tarea de autorizar usuarios a través de permisos. Se eligió `django-rest-framework` para crear una *API REST* para que otros servicios puedan consultar o crear datos. Usando `django-rest-framework-simplejwt`, se implementó *JWT*. Es decir, todo servicio que quiera acceder a los datos tendrá que pedir un *token*. Si éste está habilitado para consumir datos, se le proveerá de dicho token.

4.2.2. Administración de los datos

Los usuarios que tengan permiso para ingresar al **Administrador**, pueden hacerlo mediante una pantalla de Login como muestra la Figura 4.3. Una vez ingresado, se puede ver la pantalla principal del administrador, como muestra la Figura 4.4, donde existen diferentes menús para utilizar los datos. Tienen acceso a diferentes listados (Figura 4.6), creación, edición y borrado (Figura 4.7). Los usuarios tienen también la posibilidad de importar planillas de datos (Figura 4.5).

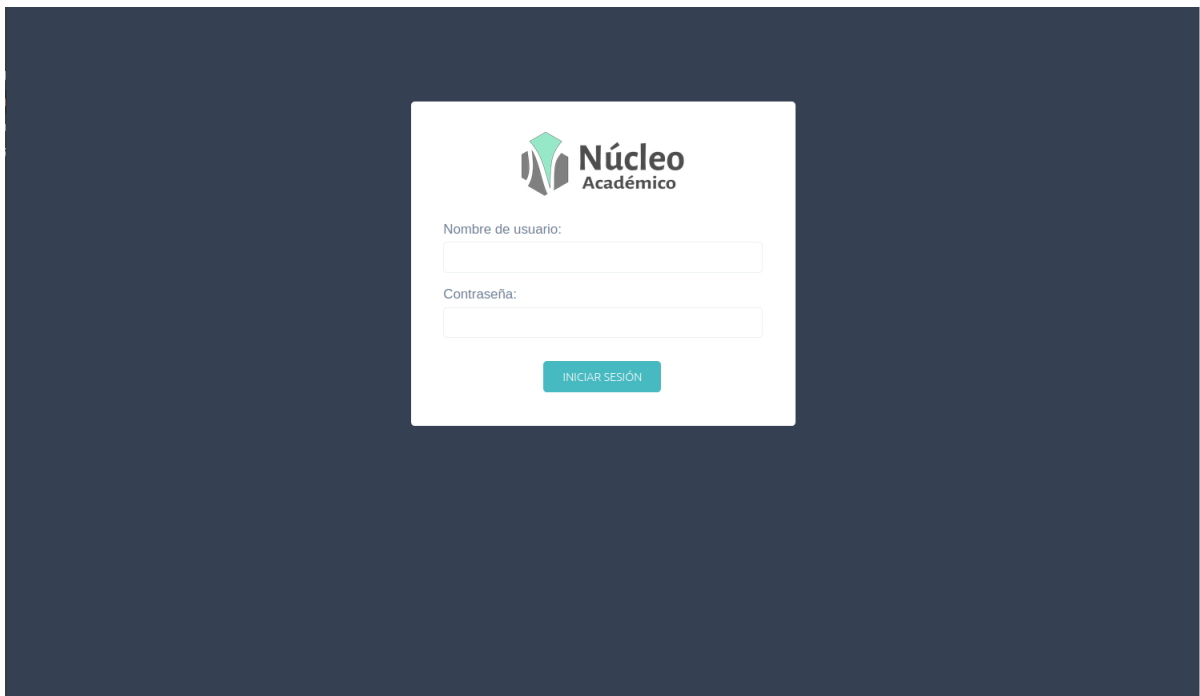


Figura 4.3: Pantalla de login

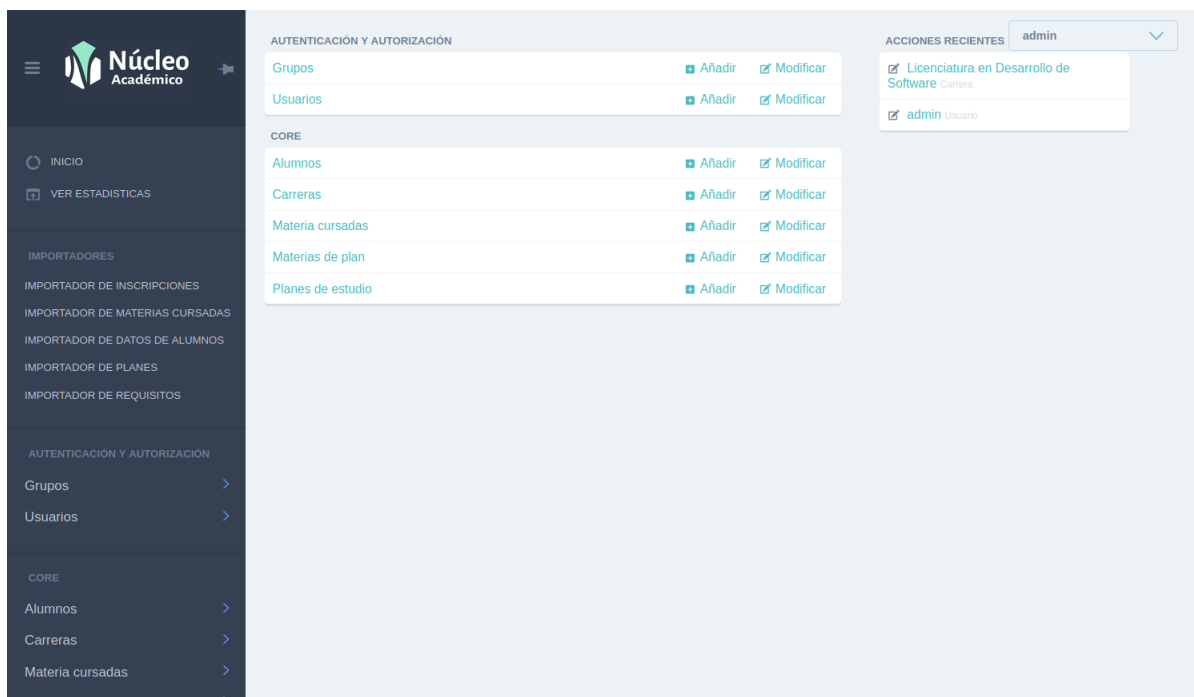


Figura 4.4: Pantalla principal del admin



Figura 4.5: Pantalla de importadores

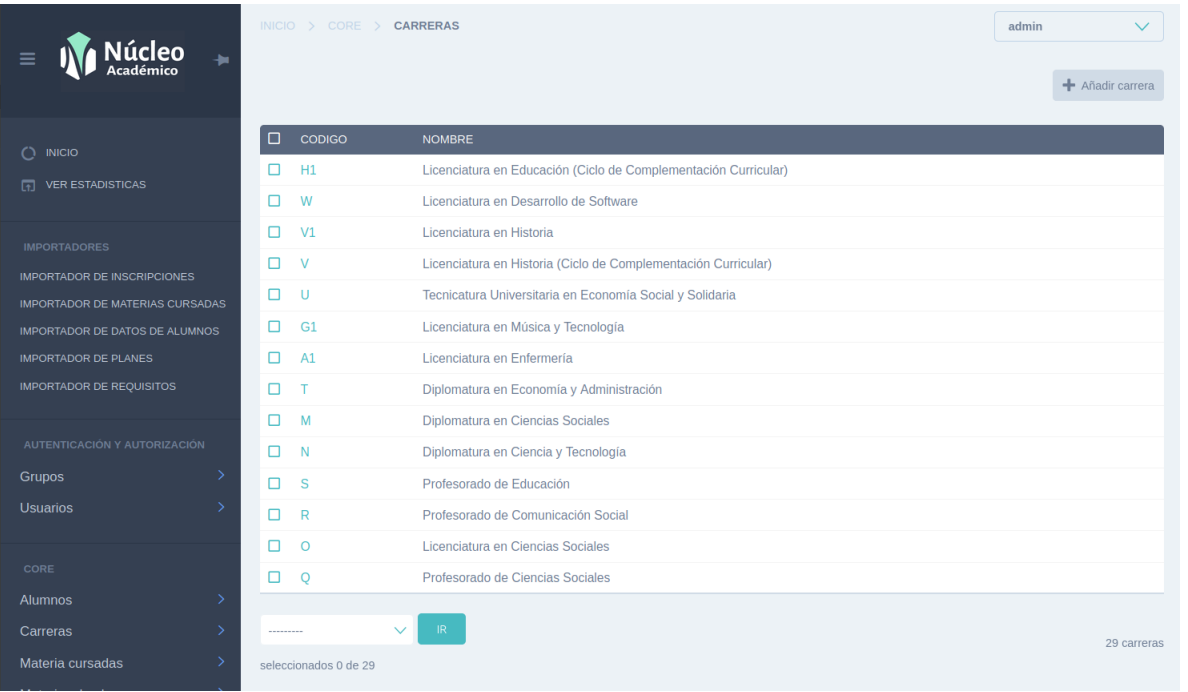


Figura 4.6: Pantalla de listados

The screenshot shows the 'Licenciatura en Desarrollo de Software' edit page. The left sidebar contains the 'Núcleo Académico' logo and a menu with categories: 'INICIO', 'VER ESTADÍSTICAS', 'IMPORTADORES' (with sub-items for inscriptions, courses, students, plans, and prerequisites), 'AUTENTICACIÓN Y AUTORIZACIÓN' (with sub-items for groups and users), and 'CORE' (with sub-items for students, careers, courses, and plans). The main content area has a breadcrumb trail: 'INICIO > CORE > CARRERAS > LICENCIATURA EN DESARROLLO DE SOFTWARE'. A user dropdown shows 'admin'. Below the breadcrumb is a 'Licenciatura ...' link and a 'Histórico' button. The main form has three tabs: 'GENERAL' (selected), 'PLANES DE ESTUDIO', and 'POSTULANTES'. The 'GENERAL' tab contains three fields: 'Nombre:*' with value 'Licenciatura en Desarrollo de Software', 'Codigo:*' with value 'w', and 'Fecha creacion:*' with value '01/01/2000' and a calendar icon. At the bottom are four buttons: 'GRABAR' (green), 'Grabar y añadir otro' (light blue), 'Grabar y continuar editando' (light blue), and 'ELIMINAR' (red with a close icon).

Figura 4.7: Pantalla de edición

4.2.2.1. Persistencia de los datos

Para la persistencia de los datos se decidió utilizar *Postgresql*. *Postgresql* es un sistema de gestión de bases de datos relacional y de código abierto, publicado bajo la licencia *Postgresql*, similar a la *BSD* o la *MIT*. Esta decisión también fue tomada dado que *Django* trae integrado un *ORM* que facilita la consulta, el guardado, la edición y el borrado de elementos en una base de datos relacional.

4.2.2.2. Usuarios y permisos

Para resolver la autenticación y autorización en el administrador de contenidos, se usó el sistema de autenticación que viene *embebido* en *Django*. Este sistema provee una manera de asignar permisos a usuarios específicos o grupos de usuarios. Estos permisos se dividen en:

- El acceso a ver objetos está limitado a los usuarios que tengan los permisos *view* o *change*.
- El acceso a ver los formularios para agregar elementos, está limitado a los usuarios que tengan el permiso *add* para ese tipo de objetos.

- El acceso a ver los listados, ver los formularios de edición y la posibilidad de editar, están limitados a los usuarios que tengan el permiso *change* para ese objeto en particular.
- El acceso a eliminar un objeto está limitado a los usuarios que tengan el permiso *delete*.

4.2.2.3. Grupos de usuarios

Django provee una forma de categorizar usuarios a los que se les puede aplicar permisos llamado *grupo*. Un usuario puede pertenecer a muchos grupos. Un usuario que pertenezca a un grupo, automáticamente tiene los permisos asignados a dicho grupo. Otra particularidad, además de los permisos, es que pueden servir para extender las funcionalidades. Por ejemplo, si quisiera que un grupo fuese “*Usuarios de LI*”, podría asignarle a ese grupo la carrera “*LI*” para que sólo pudieran ver datos relacionados a su carrera y no a otras.

4.2.3. Importadores

En el caso que los usuarios tengan la necesidad de importar los datos a través de planillas *.ods*, se realizaron diferentes importadores para esta tarea (Figura 4.5). Estos importadores son:

- Carreras
- Planes de estudio con sus materias
- Prerrequisitos obligatorios y recomendados de materias
- Estudiantes con sus datos personales
- Materias cursadas por estudiantes
- Inscripciones a materias

Las planillas *.ods* como input de los importadores se obtienen a partir pedidos explícitos de la Dirección de la Carrera a la Secretaría de Gestión Académica, que es la oficina responsable de la administración de Guaraní.

4.2.4. API

Se diseñó una *API* para consultar datos a través de distintas *URIs* (Tabla 4.2), para las cuales se necesita de un *token* (Tabla 4.1).

URI	Método	Parámetros	Content-Type
api/token/	POST	username,password	application/x-www-form-urlencoded

Tabla 4.1: Método para pedir un token al núcleo

URI	Método	Autorización
carreras/<str:cc>/alumnos/	GET	Bearer
carreras/<str:cc>/alumnos-completos/	GET	Bearer
carreras/<str:cc>/planes/<int:plan_año>/	GET	Bearer
carreras/<str:cc>/planes/<int:pa>/cantidad-materias-necesarias/	GET	Bearer
carreras/<str:cc>/planes/	GET	Bearer
carreras/<str:cc>/materiascursadas/	GET	Bearer
carreras/<str:cc>/inscripciones/	GET	Bearer
carreras/<str:cc>/cantidad-graduados/	GET	Bearer
carreras/<str:cc>/cantidad-graduados/<int:año>/	GET	Bearer
carreras/<str:cc>/cantidad-cursantes/	GET	Bearer
carreras/<str:cc>/cantidad-cursantes/<int:año>/	GET	Bearer
carreras/<str:cc>/cantidad-ingresantes/	GET	Bearer
carreras/<str:cc>/cantidad-ingresantes/<int:año>/	GET	Bearer
carreras/<str:cc>/cantidad-postulantes/<int:año>/	GET	Bearer
carreras/<str:cc>/cantidad-postulantes/	GET	Bearer
alumno/<str:legajo>/cursadas/	GET	Bearer
alumno/<str:legajo>/inscripciones/	GET	Bearer
materia/<str:codigo>/alumnos/	GET	Bearer

Tabla 4.2: Métodos para pedir datos. (cc: código de carrera, pa: plan año)

El *header* del *token* tiene la siguiente información:

```

1  {
2    "typ": "JWT",
3    "alg": "HS256"
4  }
```

Para el caso del *payload*, se tiene la siguiente información indicando precisamente qué carreras puede consultar el usuario:

```

1  {
2    "token_type": "access",
3    "exp": 1589296602,
4    "jti": "a1032b6c7b7f4f9d8797244ebe41a99a",
5    "user_id": 1,
6    "carreras": [
```

```
7      "W"  
8    ],  
9    "username": "admin"  
10  }
```

4.2.4.1. Ejemplo de pedido

Si se quisiera saber qué materias se definen dentro de una carrera en un plan determinado, se debería hacer de la siguiente manera:

```
GET 'https://url.del.nucleo/api/carreras/W/planes/2015/'  
--header Authorization: Bearer <token>
```

El resultado tendrá la siguiente forma:

```
1  [{  
2      "id": 54,  
3      "materia": "Taller de Trabajo Intelectual",  
4      "plan": 2015,  
5      "nucleo": "",  
6      "creditos": 4,  
7      "area": "Taller",  
8      "codigo": "00751"  
9  },  
10  ...  
11  ]
```

La Figura 4.8 muestra al núcleo y un servicio que intenta hacer pedidos. Primero deberá enviar un pedido con sus credenciales para luego obtener un jwt. Una vez obtenido el jwt, puede hacer pedidos normalmente con el *token* en el *header* de cada *request*.

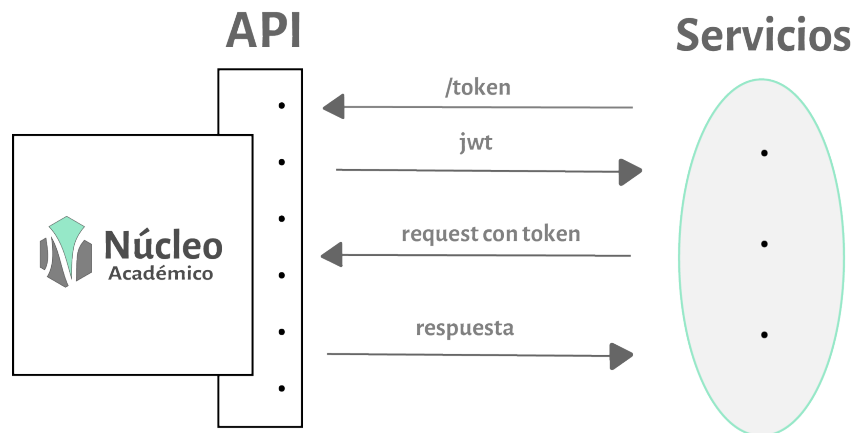


Figura 4.8: Funcionamiento del pedido de token y posteriores requests

4.2.5. Tests

Se diseñaron e implementaron diferentes tests de unidad utilizando la biblioteca de Python llamada PyTest. Estos tests poseen una anatomía tradicional, con los siguientes cuatro pasos:

- **Organización:** es la preparación inicial para el test. Esto conlleva toda tarea anterior a la acción, para que en esa etapa se puedan realizar todas las tareas en un solo paso. En esta etapa se definieron los objetos relacionados con carreras, planes de estudio, estudiantes, credenciales y usuarios con sus perfiles.
- **Assert:** En esta etapa se realizaron las pruebas en sí, tanto para comportamiento de distintos cálculos que se realizan en el núcleo como el comportamiento de la API (con mucho énfasis en la autenticación y autorización).
- **Limpieza:** esta etapa es donde se limpia el test luego de correr, para que las siguientes pruebas no estén influenciadas por este último.

4.2.5.1. Test Coverage

Test Coverage es una métrica que determina porcentualmente la cantidad de código que fue probado. Aunque este porcentaje no determina la calidad de las pruebas, es útil para determinar qué código no fue alcanzado por tests.

Para esta tarea se utilizó la herramienta Coverage, la cual determinó que se alcanzó un 87,44 % de código alcanzado por tests (Figura 4.9).

Files	≡	●	●	●	Coverage
■ alumnos	31	31	0	0	100.00%
■ core	530	440	0	90	83.02%
■ tests	241	230	0	11	95.44%
📄 manage.py	2	2	0	0	100.00%
Folder totals (17 files)	804	703	0	101	87.44%
Project totals (17 files)	804	703	0	101	87.44%

Figura 4.9: Test Coverage de 87,44 %

4.2.6. Integración Continua

Integración Continua es la práctica de realizar pruebas sobre el código de manera frecuente. El objetivo principal es crear software más sano mediante el desarrollo y las pruebas en incrementos mas pequeños. Para esta tarea se decidió usar Travis CI.

Como plataforma de integración continua, Travis CI realiza el *build* del proyecto y corre los tests de forma automática, al detectar cambios nuevos en el repositorio GitHub. En caso de detectar algun error, ya sea en la etapa de *build* o en los tests, Travis notifica a todos los usuarios del proyecto.

Para el núcleo, se configuró para generar la imagen de Docker y luego correr los tests, enviando los resultados a Coverage.

4.2.7. Deploy

Las aplicaciones fueron desplegadas en un servidor de la Universidad Nacional de Quilmes. Dicho servidor cuenta con el sistema operativo Debian 10. Con respecto al hardware, cuenta con un procesador de 4 nucleos de 2.4GHZ, 4GB de memoria RAM y 20GB de disco rígido.

4.2.7.1. Repositorios

El código y las configuraciones quedaron alojadas en *GitHub* de forma libre como cinco repositorios distintos. Estos repositorios se corresponden al núcleo, el análisis, la visualización, la configuración de *Nginx* y las configuraciones para hacer *deploy* del *stack* completo de *Docker Swarm*. Cada proyecto cuenta con la documentación correspondiente para desplegarse de forma independiente, o para desplegar el *stack* completo.

A su vez, existen cuatro repositorios de imágenes de *Docker* en *Docker Hub*. Estas imágenes se corresponden al núcleo, el análisis, la visualización y *Nginx*.

4.2.7.2. DockerSwarm

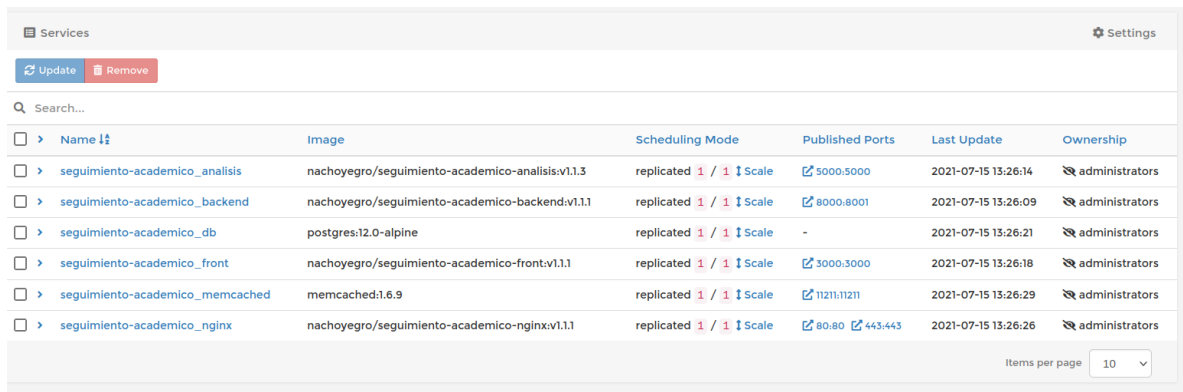
Si bien en un principio se optó por un *deploy* utilizando *docker-compose*, esta acción llevaría a una tarea manual a la hora orquestar los contenedores. Por esta razón, se optó por un *deploy* utilizando *Docker Swarm* y *Portainer* para su administración.

Dentro del proyecto *seguimiento-stack*, se debe correr el siguiente comando:

```
docker stack deploy --compose-file seguimiento-stack.yml
seguimiento-academico --with-registry-auth
```

Esto, además de crear todos los servicios, se encarga de crear la red *seguimiento-academico_default*, la cual comparten todos los contenedores y pueden interactuar a través de ésta.

Luego de esto, se puede ingresar a *Portainer* (por defecto corre en el puerto 9000 del *host*) para administrar estos contenedores (Figura 4.10).



Name	Image	Scheduling Mode	Published Ports	Last Update	Ownership
seguimiento-academico_analisis	nachoyegro/seguimiento-academico-analisis:v1.1.3	replicated 1 / 1 Scale	5000:5000	2021-07-15 13:26:14	administrators
seguimiento-academico_backend	nachoyegro/seguimiento-academico-backend:v1.1.1	replicated 1 / 1 Scale	8000:8001	2021-07-15 13:26:09	administrators
seguimiento-academico_db	postgres:12.0-alpine	replicated 1 / 1 Scale	-	2021-07-15 13:26:21	administrators
seguimiento-academico_front	nachoyegro/seguimiento-academico-front:v1.1.1	replicated 1 / 1 Scale	3000:3000	2021-07-15 13:26:18	administrators
seguimiento-academico_memcached	memcached:1.6.9	replicated 1 / 1 Scale	11211:11211	2021-07-15 13:26:29	administrators
seguimiento-academico_nginx	nachoyegro/seguimiento-academico-nginx:v1.1.1	replicated 1 / 1 Scale	80:80 443:443	2021-07-15 13:26:26	administrators

Figura 4.10: Administración de contenedores desde Portainer

4.3. Análisis y Visualización de Datos

4.3.1. Análisis de datos

Para poder obtener información relevante de las carreras, materias y estudiantes, se debe hacer un análisis de los datos recopilados históricamente.

Si bien el núcleo provee una gran cantidad de datos, éstos necesitan una manipulación, procesamiento y una limpieza para que puedan ser analizados.

El objetivo principal de este módulo es de la obtención de los datos mediante una *API Rest*, la manipulación y el procesamiento de esos datos. Una vez terminado este proceso, sirve los resultados mediante una *API Rest*, que es consumida por un tercero para su visualización.

4.3.1.1. Especificaciones

El módulo de análisis de datos se implementó con *Python 3.6*. El análisis de los datos se implementó con *Pandas* y los resultados son servidos gracias a *Flask* a través de una *API REST*.

La Figura 4.11 muestra el rol de este módulo y su funcionamiento con respecto al núcleo

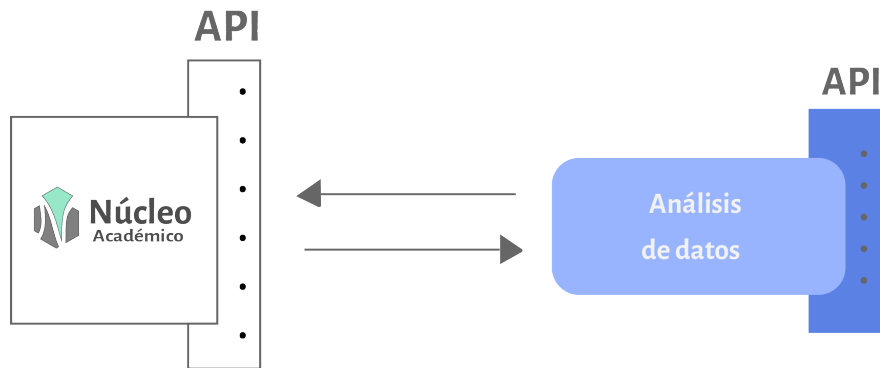


Figura 4.11: Obtención de datos a través del núcleo

4.3.1.2. Obtención de los datos

Los datos se obtienen a través de pedidos a la *API* del Núcleo. Estos datos llegan en forma de *JSON* y se transforman a *DataFrame* o *Series* (estructuras de datos de *Pandas*).

Dado que los datos llegan de forma de recursos independientes, la mayor parte de los análisis requieren que estos datos sean unidos gracias a las funcionalidades de los *DataFrames* (de forma similar a los *join* de *SQL*).

4.3.1.3. Tests

Como en el caso del Núcleo, se diseñaron e implementaron diferentes tests de unidad utilizando la biblioteca de Python llamada *PyTest*.

4.3.1.4. Test Coverage

Para esta tarea se utilizó la herramienta Coverage, la cual determinó que se alcanzó un 90,1 % de código alcanzado por tests (Figura 4.12).

Files	≡	●	●	●	Coverage
alumnos	31	31	0	0	100.00%
core	530	440	0	90	83.02%
tests	241	230	0	11	95.44%
manage.py	2	2	0	0	100.00%
Folder totals (17 files)	804	703	0	101	87.44%
Project totals (17 files)	804	703	0	101	87.44%

Figura 4.12: Test Coverage de 90,1 %

4.3.1.5. Integración Continua

Para el análisis se configuró Travis para generar la imagen de Docker y luego correr los tests, enviando los resultados a Coverage.

4.3.2. Visualización de los datos analizados

Si bien la parte de análisis de datos quedó cubierta, era necesario poder visualizarlos de forma gráfica. Por esta razón, surgió la necesidad de crear una nueva aplicación independiente del núcleo y del análisis de datos.

Esta nueva aplicación obtuvo el nombre de *Seguimiento Académico*, y su logo se muestra en la Figura 4.13.



Figura 4.13: Logo de la aplicación de Seguimiento Académico

Su objetivo principal es recolectar los datos analizados y mostrarlos de forma gráfica, dependiendo de los permisos que tenga el usuario que realiza el pedido.

Estos datos se desprenden del módulo de análisis, y se agrupan de la siguiente manera:

- Datos sobre carreras.
- Datos sobre materias.
- Datos sobre estudiantes.

Cada uno de estos grupos de datos tiene su propia pantalla de tipo reporte, donde el usuario ingresa los parámetros necesarios para cada reporte.

4.3.2.1. Especificaciones

Seguimiento Académico está construido con *React 16.8* y se usó la biblioteca *recharts* para generar gráficos. Además, se usó el *framework Material UI* para el renderizado de elementos de la interfaz de usuario y diferentes tablas. Este módulo realiza pedidos a la aplicación de análisis de datos, los cuales tienen que tener su correspondiente *token* para ser aceptados. Cuando un usuario ingresa, se hace una autenticación con el núcleo. Esta autenticación tiene como resultado un *token*, y éste será utilizado para realizar todos los pedidos de datos correspondientes.

El gráfico 4.1 muestra el rol de este módulo y su funcionamiento con respecto al de análisis de datos y al núcleo.

4.3.2.2. Obtención de los datos

Los datos se obtienen a través de pedidos a la *API* del módulo de análisis de datos gracias a *Axios*. *Axios* está basado en *promises*, con lo cual se puede aprovechar las ventajas de *async* y *await* para un código asíncronico más legible.

Estos pedidos se realizan a través de llamadas *ajax*, lo cual quiere decir que la página carga normalmente y realiza los pedidos de forma asíncronica e independiente entre los gráficos. A medida que los datos van llegando, cada gráfico se va *renderizando*.

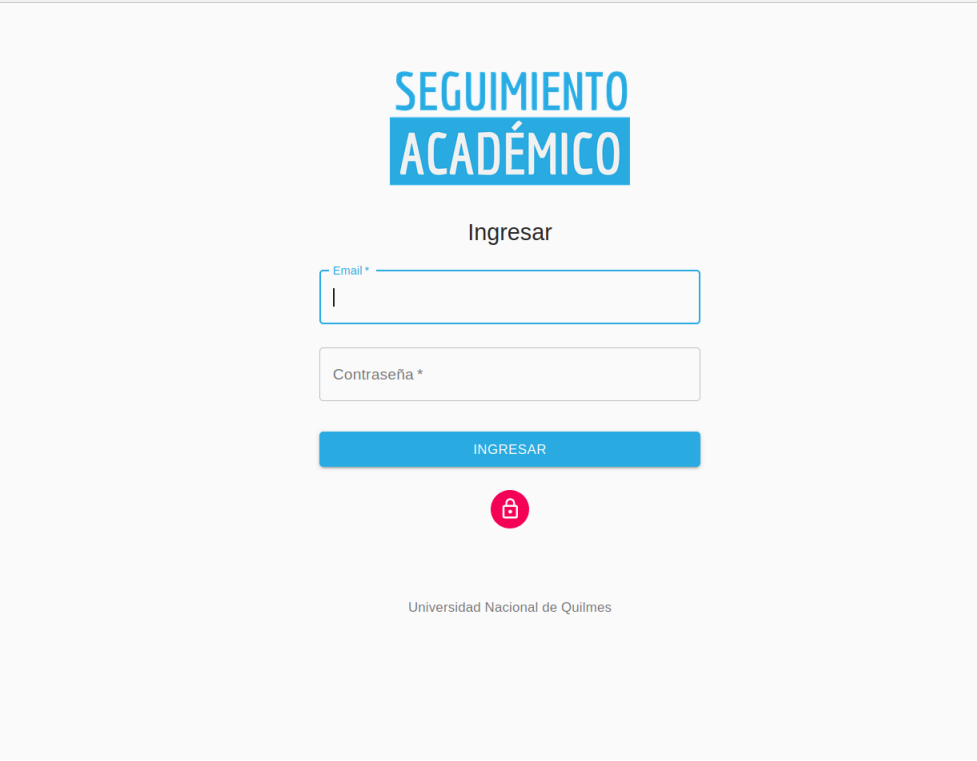
Estos datos llegan como *JSON* y se procesan para ser mostrados de forma correcta.

4.3.2.3. Pantallas principales

La aplicación *Seguimiento Académico* tiene 5 pantallas principales:

- Login (Figura 4.14).
- Home (Figura 4.15)

- Reporte de Carrera (Figura 4.16)
- Reporte de Materia (Figura 4.17)
- Reporte de Alumno (Figura 4.18)



The image shows a login interface for 'SEGUIMIENTO ACADÉMICO'. At the top, the title 'SEGUIMIENTO ACADÉMICO' is displayed in blue, with 'SEGUIMIENTO' in a sans-serif font and 'ACADÉMICO' in a bold sans-serif font inside a blue rectangular box. Below the title, the word 'Ingresar' is centered. The form consists of two input fields: the first is labeled 'Email *' and contains a single character 'I'; the second is labeled 'Contraseña *'. Below these fields is a blue button with the text 'INGRESAR' in white. Under the button is a red circular icon containing a white padlock. At the bottom, the text 'Universidad Nacional de Quilmes' is centered.

Figura 4.14: Login de Seguimiento Académico

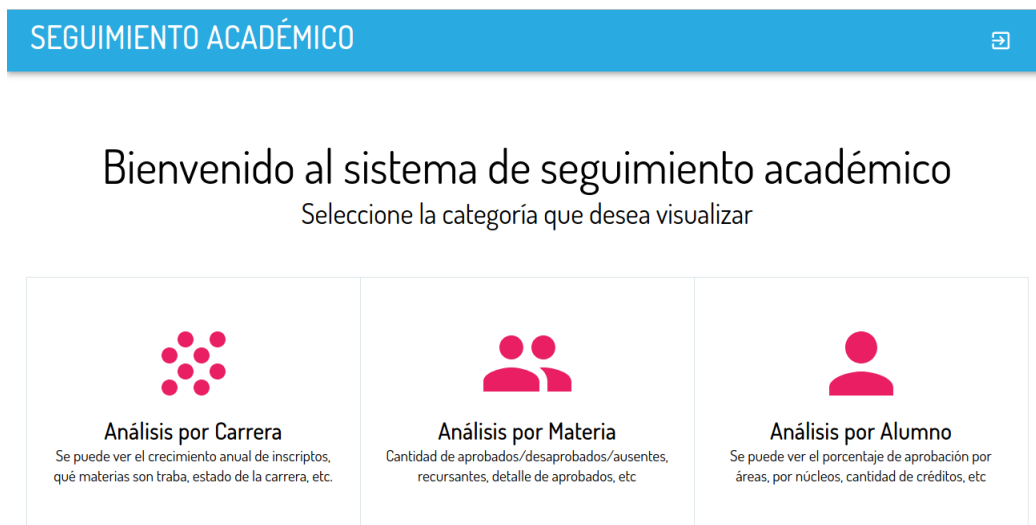


Figura 4.15: Pantalla principal de Seguimiento Académico

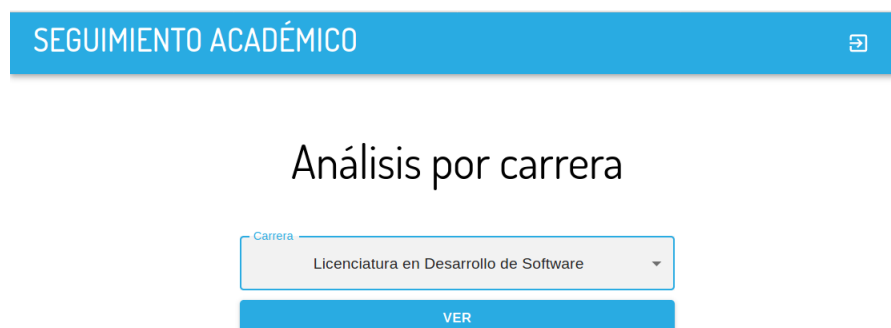


Figura 4.16: Pantalla Reporte de Carrera de Seguimiento Académico

SEGUIMIENTO ACADÉMICO

Análisis por materia

Carrera

Licenciatura en Desarrollo de Software

Materia *

01037

Fecha Inicio

01/01/2015

Fecha Fin

01/01/2020

CALCULAR ESTADISTICAS

Figura 4.17: Pantalla Reporte de Materia de Seguimiento Académico

SEGUIMIENTO ACADÉMICO

Análisis por alumno

Carrera

Licenciatura en Desarrollo de Software

Alumno *

21872

Plan *

2015

Fecha Inicio

01/01/2015

Fecha Fin

01/01/2020

CALCULAR ESTADISTICAS

Figura 4.18: Pantalla Reporte de Alumno de Seguimiento Académico

4.3.2.4. Análisis y visualización sobre materias

Datos básicos históricos

Dada una materia, se analizan cuántos estudiantes aprobaron, cuántos estudiantes desaprobaron, cuántos estudiantes quedaron ausentes y cuántos estudiantes faltan aprobarla dentro de un período de tiempo.

La Figura 4.19 muestra un gráfico en forma de columnas donde se visualiza los resultados de los estudiantes en el rango seleccionado.

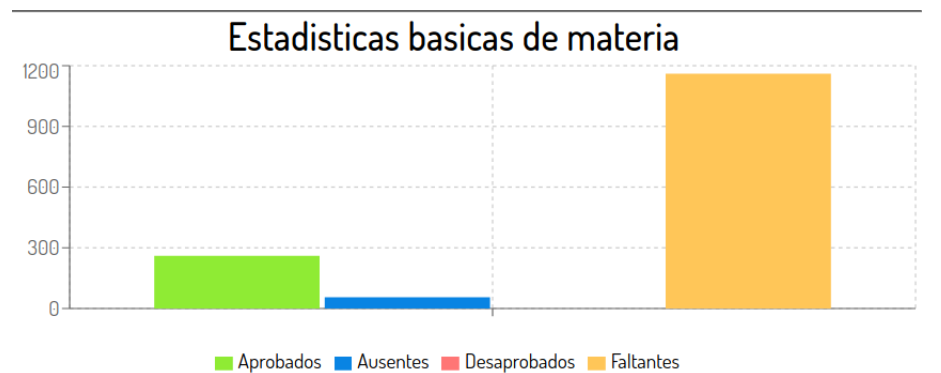


Figura 4.19: Estadísticas básicas de la materia

Estos datos se obtienen a través del módulo de análisis de datos con los siguientes datos.

```
URI: /materias/<cod\ materia>/basicos
```

Parámetros opcionales: inicio: yyyy-mm-dd, fin: yyyy-mm-dd, carrera: código de la carrera.

Detalle de aprobación

La Figura 4.20 muestra un gráfico en forma de columnas que detalla de qué forma aprobaron los estudiantes esa materia en el rango seleccionado. Se puede observar cuántos promocionaron en otra carrera, cuántos promocionaron, cuántos aprobaron un examen equivalente, cuántos con una equivalencia equivalente y cuántos con examen.

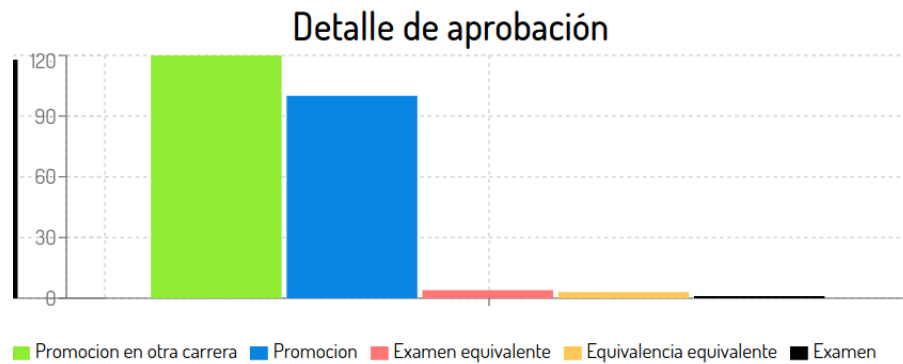


Figura 4.20: Detalle de aprobación de la materia

La forma en que obtiene los datos se hace a través de la siguiente *URI*.

URI: /materias/<cod_materia>/detalle-aprobados

Parámetros opcionales: inicio: yyyy-mm-dd, fin: yyyy-mm-dd, carrera: código de la carrera.

Las distintas formas de aprobación (como se especificaron en el Capítulo 2 son:

- Equivalencia equivalente
- Promoción en otra carrera
- Promoción
- Equivalencia
- Examen equivalente
- Examen

Dispersión de notas

Dada una materia, se analiza la dispersión de las notas dentro de un período, teniendo en cuenta estudiantes, sus notas y el promedio de ese estudiante.

La Figura 4.21 muestra de forma gráfica este análisis. El eje vertical muestra el promedio general del alumno. Por su parte, el eje horizontal muestra la nota que obtuvo.

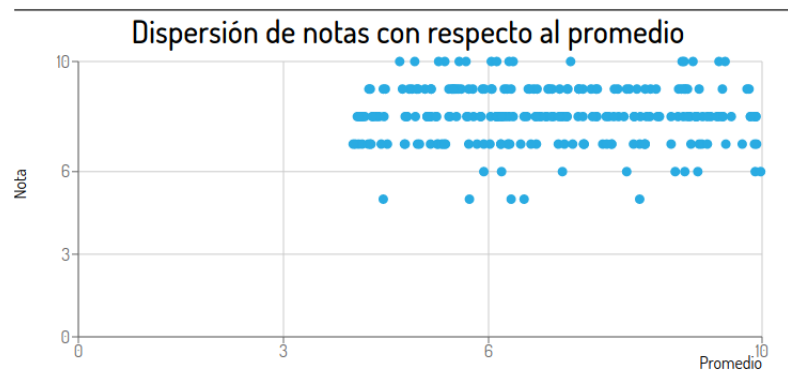


Figura 4.21: Dispersión de notas con respecto al promedio

La forma en que obtiene los datos se hace a través de la siguiente *URI*:

URI: /materias/<cod_materia>/dispersion-notas

Parámetros opcionales: inicio: yyyy-mm-dd, fin: yyyy-mm-dd, carrera: código de la carrera.

Recursantes

Dada una materia, analiza qué estudiantes son recursantes y cuántas veces.

La Figura 4.22 muestra una tabla con los recursantes dentro de una materia teniendo en cuenta las inscripciones de la fecha pedida.

Como el reporte de materias tiene fecha de inicio y fecha de fin, para este análisis se tiene en cuenta el semestre de la fecha fin. Es decir, si se pide un análisis hasta la fecha 12/12/2019, se calcularán los recursantes de la materia pedida durante las inscripciones del segundo semestre del 2019.

Recursantes		Q Buscar	×
Legajo	Veces que recursa		
31716	3		
33245	3		
36339	2		
36338	2		
37554	2		
		5 filas	< 1-5 of 11 >

Figura 4.22: Recursantes

Los datos son obtenidos a través de la *URI*:

```
URI: /materias/<cod\ materia>/recursantes
```

Parámetros opcionales: fecha: yyyy-mm-dd

4.3.2.5. Análisis sobre estudiantes

Porcentaje de aprobación por área

Dado un estudiante, se hace un análisis de los porcentajes de aprobación que tiene con respecto a las diferentes áreas de la carrera y el plan que se está analizando, dentro del período elegido.

La Figura 4.23 visualiza un gráfico de radar que muestra los porcentajes de aprobación del estudiante dentro de las áreas de la carrera.

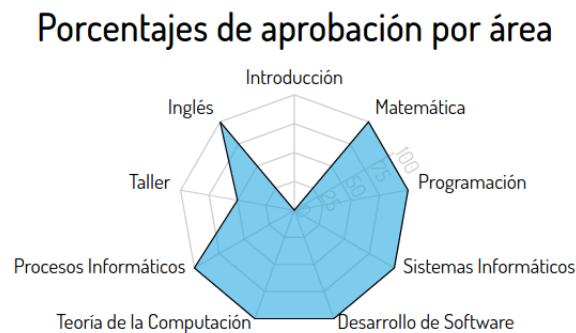


Figura 4.23: Porcentajes de aprobación por área

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

```
URI: /alumnos/<legajo>/porcentajes-areas
```

Parámetros adicionales: carrera, inicio, fin, plan

Cada carrera tiene sus propias áreas. En particular, y tomado a forma de ejemplo, las áreas de la Licenciatura en Informática se muestran en la Tabla 4.3.

Nombre
Programación
Sistemas Informáticos
Procesos Informáticos
Desarrollo de Software
Teoría de la Computación

Tabla 4.3: Áreas de la Licenciatura en Informática

Porcentaje de aprobación por núcleo

Dado un estudiante, se hace un análisis de los porcentajes de aprobación que tiene con respecto a los diferentes núcleos de la carrera y el plan que se está analizando, dentro del período elegido.

El siguiente gráfico de radar muestra los porcentajes de aprobación del estudiante dentro de los núcleos de la carrera (Figura 4.24).

Porcentajes de aprobación por núcleo

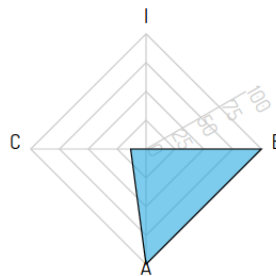


Figura 4.24: Porcentajes de aprobación por núcleo

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /alumnos/<legajo>/porcentajes-nucleos

Parámetros adicionales: carrera, inicio, fin, plan

Cada carrera tiene sus propios núcleos. En particular, y tomado a forma de ejemplo, los núcleos de la Licenciatura en Informática se muestran en la Tabla 4.4.

Código	Nombre
I	Introductorio
B	Básico
A	Avanzado
C	Complementario/Orientativo

Tabla 4.4: Núcleos de la Licenciatura en Informática

Notas

Dado un estudiante, se obtienen las diferentes notas que obtuvo a lo largo de sus cursadas. La siguiente tabla muestra las notas del estudiante dentro del rango seleccionado (Figura 4.25).

Notas del alumno			
Fecha ↑	Materia	Plan	Nota
05/06/2019	Teoría de la Computación	2015	7
02/05/2019	Algoritmos	2015	8
10/06/2018	TTU	2015	A
08/06/2018	Objetos III	2015	9
07/06/2018	Arquitectura de Software I	2015	10
Filas por página: 5 ▼ 1-5 of 6 < >			

Figura 4.25: Notas del estudiante

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /alumnos/<legajo>/notas

Parámetros adicionales: carrera, inicio, fin, plan

Porcentaje de avance de carrera

Dado un legajo de estudiante, se realiza un análisis para determinar el porcentaje de avance de la carrera, según el plan que se analice. Dado que cada plan tiene una cantidad distinta de materias, este dato cambia según el plan que se analice.

Además, este dato se analiza en base a las fechas pedidas en el reporte.



Figura 4.26: Porcentaje de avance de carrera

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /alumnos/<legajo>/porcentaje-carrera

Parámetros adicionales: carrera, inicio, fin, plan

Promedio por períodos

Dado un estudiante, se calcula cuál fue su *score* por cada período. Es decir, su rendimiento semestral.

El *score* se calcula sacando un promedio de sus notas en un semestre. Las materias que sólo se distinguen entre “Aprobado” y “Desaprobado” y no disponen de una nota, se les asigna una nota para obtener este promedio.

Las materias con nota “Aprobado” son consideradas como un 7, y las materias con nota “Desaprobado” son consideradas como un 3. Entonces, el *score* no es un promedio preciso, sino una estipulación del rendimiento de un estudiante en ese período.

Este *score* es útil para analizar el rendimiento de un estudiante y puede servirle a las carreras para tomar decisiones tempranas.

El siguiente gráfico muestra el desempeño del estudiante semestralmente dentro del rango seleccionado (Figura 4.27).

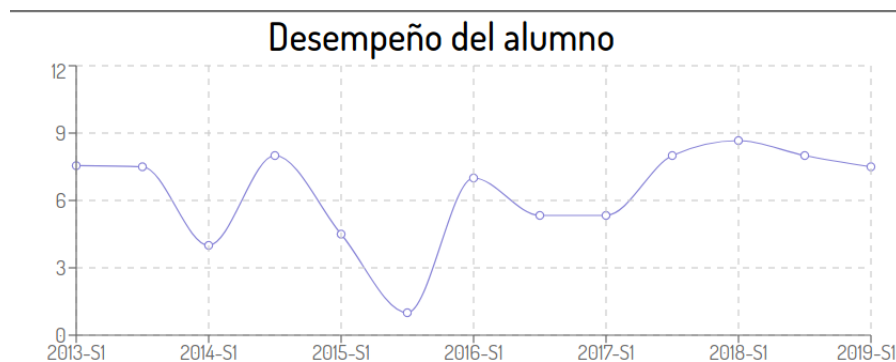


Figura 4.27: Detalle de aprobación

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

```
URI: /alumnos/<legajo>/scores
```

Parámetros adicionales: carrera, inicio, fin, plan

4.3.2.6. Análisis sobre carreras

Cantidad de estudiantes por semestre

Dada una carrera, se obtiene un listado de cantidad de inscriptos por semestre

El siguiente gráfico es en forma de *área*, y muestra la cantidad de ingresos semestrales histórico de la carrera. De esta forma, se puede observar en qué períodos hubo picos de ingresos y en qué períodos hubo una baja de ingresos (Figura 4.28).

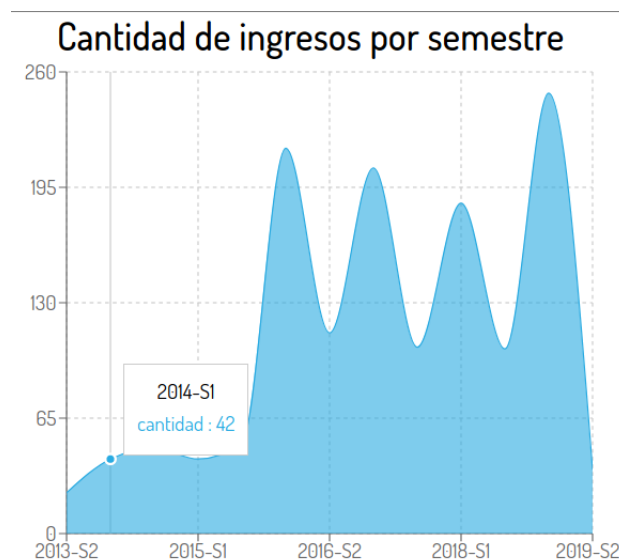


Figura 4.28: Ingresos por semestre

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

```
URI: /carreras/<carrera>/alumnos
```

Datos generales por cohorte

Dada una carrera, se obtiene un listado de inscriptos, cursantes, graduados y postulantes por cohorte.

La siguiente es una tabla que detalla año a año de la carrera, qué cantidad de cursantes hubo, la cantidad de ingresantes y la cantidad de graduados (Figura 4.29).

Alumnos por cohorte			
Cohorte ↑	Cursantes	Ingresantes	Graduados
2019	1421	350	1
2018	1071	295	0
2017	776	189	0
2016	587	101	0
2015	486	92	0

Filas por página: 5 1-5 of 6 < >

Figura 4.29: Tabla de estudiantes por cohorte

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /carreras/<carrera>/cantidades-alumnos

Ingresantes por cohorte

Dada una carrera, se obtiene la cantidad de ingresantes separadas por año.

La siguiente es una tabla que detalla año a año de la carrera, qué cantidad de ingresantes hubo (Figura 4.30).

Ingresantes por cohorte	
Cohorte	↑ Ingresantes
2019	350
2018	295
2017	189
2016	101
2015	92

Filas por página: 5 1-5 of 6 < >

Figura 4.30: Tabla de ingresantes por cohorte

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /carreras/<carrera>/cantidades-ingresantes

Cantidad de cursantes actual

Dada una carrera, se obtiene la cantidad de cursantes actual.

La cantidad actual de cursantes de la carrera se muestra en forma de tarjeta informativa (Figura 4.31).



Figura 4.31: Cantidad de cursantes

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente URI:

URI: /carreras/<carrera>/cursantes-actual

Cantidad de ingresantes actual

Dada una carrera, se obtiene la cantidad de ingresantes.

La cantidad actual de ingresantes de la carrera se muestra en forma de tarjeta informativa (Figura 4.32).



Figura 4.32: Cantidad de ingresantes

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente URI:

URI: /carreras/<carrera>/ingresantes-actual

Cantidad total de graduados

Dada una carrera, se obtiene el total de graduados.

La cantidad total de graduados de la carrera se muestra en forma de tarjeta informativa (Figura 4.33).

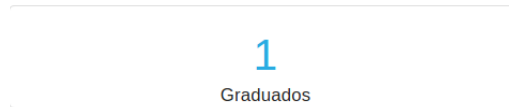


Figura 4.33: Cantidad de graduados

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /carreras/<carrera>/graduados-total

Dispersión de scores y promedios

El objetivo de este análisis es el agrupar estudiantes en base al *score* y al promedio, mostrando grupos de interés sobre los cuales se podrán implementar acciones específicas.

La Figura 4.34 muestra un gráfico con la dispersión de los estudiantes con respecto a su promedio general y su desempeño el último año. Uno de los ejes tiene en cuenta el promedio y el otro su *score* el último año. De esta manera, se puede observar de forma gráfica los estudiantes que se desempeñaron por encima de su historial y cuáles no.

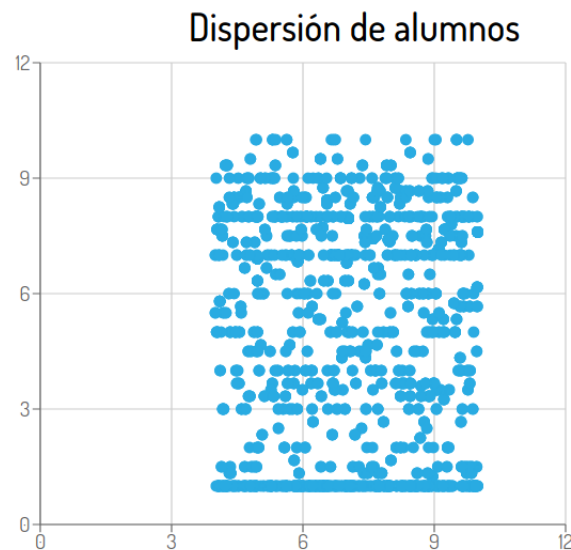


Figura 4.34: Gráfico de dispersión

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*

URI: /carreras/<carrera>/dispersion-score-promedio

Materias que demoran la formación

Dada una carrera, se quiere analizar qué materias son las que demoran la formación de los estudiantes. Para poder realizar este análisis, se necesitaron los siguientes datos:

- Índice de aprobación de una materia (cantidad de aprobados / cantidad total de cursantes).
- Cuántas materias obligatorias dependen de ella.

Luego de calcular estos datos materia por materia, se calcula un *score*. Este *score* determina qué materia puede llegar a demorar la carrera de un estudiante.

$$Score = (1 - IndiceAprobacion) * CantidadObligatoriasDependientes$$

Es decir, que una materia que tiene un porcentaje de aprobación del 75 % y 10 materias que son obligatorias y dependen de ésta para ser cursadas, tendrá un puntaje de 2.5 $((1 - 0.75) * 10)$.

La siguiente tabla muestra un puntaje por materia en base a su porcentaje de aprobación y sobre la dependencia que tienen otras materias sobre ésta para ser cursadas (Figura 4.35).

Scores de materias en base a dificultad/dependencia			
Materia ↑	Promedio de aprobación	Materias Dependientes	Score
Análisis Matemático I	79.51	2	0.41
Características de Lenguajes de Programación	76.19	1	0.24
Laboratorio de Sistemas Operativos y Redes	95.26	6	0.28
Introducción a la Programación	59.86	38	15.25
Matemática I	60	10	4

Filas por página: 5 1-5 of 6 < >

Figura 4.35: Tabla de ingresantes por cohorte

Estos datos se obtienen a través de la aplicación de análisis de datos a través de la siguiente *URI*:

URI: /carreras/<carrera>/materias-traba

4.4. Licencia

El código será licenciado bajo una licencia libre avalada por la *OSI (Open Source Initiative)* y la *FSF (Free Software Foundation)*. El código fuente será subido de forma pública a un repositorio de código online de la UNQ para el libre acceso de todo aquel que desee utilizar, estudiar, modificar, copiar, redistribuir, mezclar, publicar o sublicenciar el software en cuestión.

4.5. Conclusiones

Se desarrollaron tres aplicaciones independientes, las cuales se comunican a través de pedidos *REST*. Cada uno de estos pedidos lleva en su encabezado un *token JWT*. Dicho *token* tiene información como las carreras que tiene el usuario asignadas para ver.

En primer lugar, el Núcleo Académico brinda datos académicos a quienes tengan permisos de visualizar, y permite crear, editar y borrar a todo aquel que disponga de dichos permisos.

Gracias al Núcleo Académico fue posible crear una aplicación de análisis y otra de visualización, llegando así a cumplir con los requerimientos iniciales.

Cada una de estas aplicaciones se encuentra aislada dentro de su propio contenedor de *Docker*, y estos contenedores pueden ser incrementados a medida que sea necesario.

En el siguiente capítulo se detalla el desarrollo de las pruebas del funcionamiento de las aplicaciones con distintos parámetros de demanda.

Capítulo 5

Pruebas realizadas

5.1. Pruebas y resultados

5.1.1. Resultados esperados

El desarrollo de las aplicaciones debe responder a los requerimientos solicitados por el cliente (en este caso específico, la Dirección de Carreras del Departamento de Ciencia y Tecnología de la UNQ). Para que se cumplan estos requisitos, la implementación tiene que ser capaz de manejar todos los pedidos que recibe y responder en consecuencia.

Si bien la problemática inicial implica que sólo se consultarán datos para analizarlos y ser visualizados por los directores de las carreras de la UNQ, no hay que descartar que pueda ser extendido dado su potencial.

Para garantizar que toda la aplicación funcione de manera correcta se realizaron diferentes pruebas con escenarios cambiantes. Para todas las pruebas se eligieron 5 *URIs* y se realizaron sobre una base de datos con 1.424 estudiantes y 13.234 materias cursadas por los mismos. La arquitectura utilizada para las pruebas se asemeja a la que dispone en producción.

Se utilizó la herramienta *jMeter* para realizar los *Requests* y evaluar los tiempos de respuesta y errores, entre otras métricas.

Para medir los tiempos de respuesta, se usará *APDEX*.

APDEX (Índice de Performance de la Aplicación, por sus siglas en inglés) es un estándar abierto para medir la *performance* de una aplicación de software. Su propósito es convertir las mediciones en información sobre la satisfacción del usuario, al especificar una forma uniforme de analizar e informar sobre el grado en que el rendimiento medido cumple con las expectativas del usuario. Para realizar una medición *APDEX*, es necesario establecer un tiempo *satisfactorio* y un tiempo *tolerable*. Una vez establecidos estos valores y teniendo los resultados de la prueba, se calcula el índice:

$$Apdex = \frac{Satisfactorios + \frac{Tolerables}{2}}{Total}$$

5.1.2. Pruebas con 100 usuarios simultáneos

En primer lugar, se realizó una prueba en la cual 100 usuarios simultáneos se encuentran haciendo pedidos al núcleo, el cual está desplegado en una sola instancia de *Docker*. Esta prueba duró 5 minutos y los resultados se muestran a continuación.

5.1.2.1. Resultados generales de la prueba

Los resultados generales, como muestra la Tabla 5.1, indican que durante los 5 minutos que duró la prueba se realizaron 16.997 *Requests* en total, y el núcleo pudo responder sin ningún error.

El tiempo promedio de respuesta fue de 1043.79ms.

Nombre del Pedido	Cantidad	Errores (%)	Tiempo Promedio (ms)
Total	16977	0.00 %	1043.79
Carrera graduados	2834	0.00 %	1038.98
Carrera ingresantes	2871	0.00 %	1057.51
Carrera postulantes	2850	0.00 %	1046.61
Carreras	2793	0.00 %	1851.00
Plan Materias necesarias	5629	0.00 %	1036.10

Tabla 5.1: Pruebas realizadas con 100 usuarios simultáneos durante 5 minutos

5.1.2.2. Resultados sobre el uso de recursos durante la prueba

Para evaluar el consumo de recursos, se utilizó el comando *docker stats* que realiza una transmisión en vivo de los recursos que van consumiendo los contenedores. En promedio utilizó un 68 % del CPU y 55MB de memoria RAM (Tabla 5.2).

Contenedor	CPU	Memoria (MB)
Núcleo	68 %	55

Tabla 5.2: Recursos en promedio consumidos por el contenedor durante la prueba

5.1.2.3. APDEX de la prueba

Para determinar el APDEX de la prueba, se estableció que la *tolerancia* es de 2 segundos, y la *frustración* es de 3 segundos.

Como se explicó anteriormente, esta prueba tiene en cuenta el tiempo que tardan todos los *Requests* para luego determinar este valor.

Para esta prueba, el APDEX fue de 0.995, lo cual es casi perfecto (el valor máximo es 1) dentro de los parámetros elegidos.

La Tabla 5.3 muestra este resultado.

APDEX Total	Tolerancia	Frustración
0.995	2 seg	3 seg

Tabla 5.3: APDEX de la prueba con 100 usuarios durante 5 minutos

5.1.3. Pruebas con 300 usuarios simultáneos

Luego de la primera prueba, se realizó una segunda en la cual 300 usuarios simultáneos se encuentran haciendo pedidos al núcleo, el cual está desplegado en una sola instancia de *Docker*. Esta prueba duró 10 minutos y los resultados se muestran a continuación.

5.1.3.1. Resultados generales de la prueba

Al igual que en la prueba anterior, *jMeter* nos muestra estos resultados (Tabla 5.4).

Con un total de 20918 pedidos, el núcleo no pudo resolver en promedio un 1.37 % de los *Requests* dando como resultado el error 502.

El tiempo promedio de respuesta fue de 4409.80ms., lo cual significa un incremento considerable.

Nombre del Pedido	Cantidad	Errores (%)	Tiempo Promedio (ms)
Total	20918	1.37 %	4409.80
Carrera Cantidad graduados	4175	1.20 %	4242.01
Carrera Cantidad ingresantes	4309	1.25 %	4461.21
Carrera Cantidad postulantes	4242	1.65 %	4641.71
Carreras	4070	1.25 %	4274.56
Plan Cantidad materias necesarias	4122	1.50 %	4420.89

Tabla 5.4: Pruebas realizadas con 300 usuarios simultáneos durante 10 minutos

5.1.3.2. Resultados sobre el uso de recursos durante la prueba

Como muestra la Tabla 5.5, hubo un incremento del uso promedio de CPU, aunque el uso de memoria se mantuvo igual.

Contenedor	CPU	Memoria (MB)
Núcleo	88 %	55

Tabla 5.5: Recursos en promedio consumidos por el contenedor durante la prueba

5.1.3.3. APDEX de la prueba

Luego de ver el tiempo promedio de respuesta de los *Requests* es esperable que el *APDEX* baje considerablemente.

En este caso fue de 0.361 (Tabla 5.6), lo que significa que esa magnitud de usuarios haciendo pedidos no genera conformidad dentro de los parámetros establecidos para la satisfacción de los usuarios.

APDEX Total	Tolerancia	Frustración
0.361	2 seg	3 seg

Tabla 5.6: APDEX de la prueba con 300 usuarios durante 10 minutos y una instancia de la aplicación

5.1.4. Pruebas con 300 usuarios simultáneos con dos instancias

Luego de la prueba anterior, donde se refleja una baja considerable del tiempo de respuesta al recibir esa magnitud de usuarios simultáneos, se realizó la misma prueba haciendo uso del balanceador de carga que provee *Nginx* y agregando otra instancia de *Docker*. Esta prueba duró 10 minutos y los resultados se muestran a continuación.

5.1.4.1. Resultados generales de la prueba

Al igual que en las pruebas anteriores, se tuvo en cuenta los resultados provistos por *jMeter* (Tabla 5.7).

Estos resultados muestran un incremento en la cantidad de pedidos que pudo resolver el núcleo, pasando de los 20.918 pedidos a 33.499.

Estos resultados también muestran una mejora en el tiempo promedio de respuesta, bajando de 4409.80ms a 2796.50ms. Además, muestran una baja en el promedio de errores pasando de 1.37 % a 0.02 %.

Nombre del Pedido	Cantidad	Errores (%)	Tiempo Promedio (ms)
Total	33499	0.02 %	2796.50
Carrera Cantidad graduados	6691	0.01 %	2844.47
Carrera Cantidad ingresantes	6828	0.04 %	2709.15
Carrera Cantidad postulantes	6756	0.04 %	2945.30
Carreras	6584	0.02 %	2782.27
Plan Cantidad materias necesarias	6640	0.00 %	2700.69

Tabla 5.7: Pruebas realizadas con 300 usuarios simultáneos durante 10 minutos

5.1.4.2. Resultados sobre el uso de recursos durante la prueba

La Tabla 5.8 muestra que el consumo de CPU por instancia bajó de 88 % a 67 % y 68 % para cada una de las respectivas instancias.

El consumo de memoria se mantuvo en los mismos parámetros, aunque al ser dos instancias el consumo se duplicó. Igualmente, son valores bajos con respecto a la arquitectura provista.

Contenedor	CPU	Memoria (MB)
Núcleo-1	67 %	54
Núcleo-2	68 %	54

Tabla 5.8: Recursos en promedio consumidos por los contenedores durante la prueba

5.1.4.3. APDEX de la prueba

APDEX Total	Tolerancia	Frustración
0.667	2 seg	3 seg

Tabla 5.9: APDEX de la prueba con 300 usuarios durante 10 minutos y dos instancias de la aplicación

5.2. Conclusiones sobre las pruebas

Luego de realizar diferentes pruebas al núcleo, se observa que con 100 usuarios simultáneos haciendo *Requests* constantes durante 5 minutos, llegando a un total de 16977 pedidos, la aplicación respondió a la perfección llegando a un APDEX de 0.995 y 0 % de errores.

Después, se planteó un escenario donde 300 usuarios simultáneos realizaron *Requests* constantes durante 10 minutos (20918 pedidos) y se pudo observar una

baja considerable del *APDEX* llegando a 0.361. Los errores también aumentaron llegando a 1.37 %.

Por último, y para mejorar los resultados para una mejor experiencia de los usuarios, se realizó la misma prueba duplicando las instancias de *Docker* y haciendo uso del balanceador de carga de *Nginx*. Los resultados mejoraron notablemente llegando a un *APDEX* de 0.667 y bajando los errores a 0.02 %. Además, se incrementó la cantidad de pedidos que resolvió en ese tiempo, llevándolos a 33499 (un 60.1 % más).

Teniendo en cuenta que en la UNQ los momentos en que los sistemas académicos son más exigidos es en el inicio y en el final de los cuatrimestres, se podría duplicar las instancias de *Docker* en estos momentos (o triplicar, de ser necesario).

Capítulo 6

Conclusiones

6.1. Conclusiones

A lo largo de esta tesis se han analizado e utilizado diversas herramientas de software para implementar una solución para la centralización, disposición, análisis y visualización de datos académicos.

En primer lugar, se diseñó una solución como servicios independientes, intercambiables y escalables según la necesidad. Esto facilita que si en un futuro aparece una tecnología superadora para una tarea en particular, sólo se tenga que reemplazar ese servicio.

Se ha logrado crear un Núcleo que almacena datos y los entrega a través de una *API REST*, la cual puede ser consumida por usuarios que tengan los permisos correspondientes.

Se creó un servicio para el análisis de los datos consultados al núcleo, el cual también entrega el resultado del procesamiento de esos datos para que puedan ser consumidos.

Luego, se creó un servicio que facilita la visualización de los datos procesados. Este servicio consume los datos del módulo de análisis y muestra los resultados en diferentes gráficos que facilitan el análisis de los datos.

Este conjunto de servicios les permite a las partes interesadas consultar la información, modificarla, consultar métricas sobre carreras, materias y estudiantes de forma individual o colectiva y consultar datos sobre períodos en particular. Esta solución se ajusta perfectamente a la arquitectura provista por la UNQ.

Se ha aprendido acerca de la importancia de la disposición de los datos y cómo esto puede ayudar a tomar decisiones tempranas sobre la vida académica de los estudiantes de la UNQ.

6.2. Lineas de Trabajos futuros

Dado que el sistema se diseñó y desarrolló de forma tal que se pueda extender fácilmente, y que sus datos puedan ser consultados por otros servicios, existen varias mejoras y puntos de extensión que se detallan a continuación.

6.2.1. Extensión de análisis de datos

Con el tiempo surgirán nuevos requerimientos para extender el análisis de datos de este trabajo. Algunos análisis que quedaron fuera de este trabajo pero serían interesantes, por ejemplo, es poder predecir las inscripciones en base a la información que se tiene de los alumnos. Además, sería interesante determinar el cumplimiento de prerrequisitos según el plan de estudios analizado sobre un alumno determinado.

6.2.2. Aplicación para estudiantes

Podría existir una aplicación para que usen los estudiantes (puede ser móvil o web), donde puedan consultar su historial dentro de la Universidad. Si bien esta tarea se puede realizar en Guaraní, se podría integrar con servicios que brinda la UNQ, como la consulta del saldo disponible en la tarjeta de usuario, notificaciones con novedades y noticias que estén relacionadas únicamente con los estudiantes. Esta aplicación, a su vez, puede tener un aspecto más social: que los estudiantes pongan recomendaciones sobre las materias. Además, podría recibir *feedback* sobre los que ya cursaron, los horarios, las comodidades y la calidad de enseñanza.

6.2.3. Integración con Guaraní

Sería muy conveniente poder conectar el Núcleo con el módulo *Guaraní* a través de una *API web*. En la actualidad no existe dicha *API*, pero con el lanzamiento de *Guaraní 3* debería ser posible. De esta forma, no se deberían importar más los datos a través de planillas, sino que se haría de forma automática a través del Núcleo.

6.2.4. Integración con sistema de encuestas de inscripción

El sistema actual que utilizan algunas carreras de la UNQ para consultar las posibles inscripciones de los estudiantes debería ser integrado al núcleo para que consuma datos, y luego debería enviar los resultados para luego con esa información mostrar métricas a los directores de carreras.

6.2.5. GraphQL en el núcleo

GraphQL es un lenguaje de *queries* para *APIs*, donde los clientes definen precisamente qué datos quieren en lugar de recibir todos los datos disponibles. Esto significaría una reducción en el tamaño de respuesta de la *API* del núcleo.

6.2.6. Proveedor de identidad (*IDP*)

Dado que el potencial de extensión es amplio, una buena forma de desligar las responsabilidades al núcleo es crear un proveedor de identidad. En lugar de los usuarios identificarse ante el núcleo y que éste les provea un *token*, debería hacerlo un servicio independiente que tenga conocimiento de todos los usuarios, y éste se encargue de validar quién puede ingresar y quién no, y qué permisos tiene.

6.2.7. Acceso Centralizado

Una vez que exista el *IDP*, sería muy útil un acceso centralizado. Esto significa que, una vez que el usuario se identifica con el *IDP*, éste lo considera **logueado** en todas las aplicaciones que el usuario tiene acceso. Cuando ingrese a otra aplicación, automáticamente el *IDP* lo deja ingresar.

Bibliografía

- [Amr17] Aymen El Amri. *Painless Docker*. Aymen El Amri, 2017.
- [AX] HTTP requests using Axios.
- [BAS] The Base16, Base32, and Base64 Data Encodings.
- [BPR] 10 Best Practices for better RESTful API.
- [Cop19] Flavio Copes. *React Handbook*. Flavio Copes, 2019.
- [COR] Understanding CORS.
- [Cor15] Stephen Corona. *Nginx: a practical guide to high performance*. O’ Reilly, 2015.
- [Dj] Documentación oficial de Django.
- [ECD] Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA).
- [EP16] Manuela Ekowo and Iris Palmer. The promise and peril of predictive analytics in higher education. pages 2–7, 10 2016.
- [Fie00] R T Fielding. *Architectural styles and the design of network-based software architectures*. PhD thesis, 03 2000.
- [Fsc] Documentación oficial de Flask.
- [GGS⁺15] Pascal Giessler, Michael Gebhart, Dmitrij Sarancin, Roland Steinegger, and Sebastian Abeck. Best practices for the design of restful web services. *Proceedings - International Conference on Software Engineering*, 10:392–, 11 2015.
- [Gri14] Miguel Grinberg. *Flask Web Development: Developing Web Applications With Python*. O’ Reilly, 2014.
- [HMA] HMAC: Keyed-Hashing for Message Authentication.

- [HTT] Rfc 2616, hypertext transfer protocol – http/1.1.
- [IDC] The Digital Universe of Opportunities: Rich Data and the Increasing Value of the Internet of Things.
- [JWE] JSON Web Encryption (JWE).
- [JWS] JSON Web Signature (JWS).
- [Mck11] Wes Mckinney. pandas: a foundational python library for data analysis and statistics. *Python High Performance Science Computer*, pages 1–8, 2011.
- [McK12] Wes McKinney. *Python for Data Analysis, 2nd Edition*. O’ Reilly, 2012.
- [New12] A Newton. Using json in ietf protocols. *The IETF Journal*, 8:18–20, 10 2012.
- [New15] Sam Newman. *Building Microservices*. O’ Reilly, 2015.
- [Non17] Julian Nonino. *Procesamiento de Datos en Tiempo Real Utilizando Docker, Zookeeper, Kafka y Storm*. PhD thesis, 03 2017.
- [Pey18] Sebastián Peyrott. *JWT Handbook*. Auth0, 2018.
- [Pou18] Nigel Poulton. *Docker Deep Dive*. Nigel Poulton, 2018.
- [Res] How rest replaced soap on the web: What it means to you.
- [RSA] RSA Cryptography Specifications Version 2.2.
- [Rub17] Daniel Rubio. *Beginning Django: Web Application Development and Deployment with Python*. 01 2017.
- [SHA] US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF).
- [She20a] Esther Shein. How universities deploy student data. *Commun. ACM*, 63(4):18–20, 2020.
- [She20b] Esther Shein. How universities deploy student data. *COMMUNICATIONS OF THE ACM*, 55:18–20, 04 2020.
- [Tan02] Andrew Tanenbaum. *Computer Networks*. 01 2002.
- [TS20] Nimit Thaker and Abhilash Shukla. Python as multi paradigm programming language. *International Journal of Computer Applications*, 177:38–42, 01 2020.

- [Vin08] S Vinoski. Restful web services development checklist. *Internet Computing, IEEE*, 12:94–96, 06 2008.
- [WAP] Web API Design - Crafting Interfaces that Developers Love.